

LOK JAGRUTI KENDRA UNIVERSITY



Master of Computer Applications (Integrated)

6th Semester

Portify

(Automated Portfolio Generator)

GROUP NO:

IMCA_Sem6_UDP_Group30

Internal Guide:

Prof. Himanshu Sharma

Project By:

Vaghela Purvarajsinh D-54 (23018501210273)



LJ University
University with a Difference

CERTIFICATE

*This is to certify that Mr. Vaghela Purvarajsinh with Enrollment number **23018501210273** Semester 10th has successfully completed his **Project** from the department of **Master of Computer Applications (Integrated)** during the academic year 2025 - 2026.*

Internal Guide name

Prof. Himanshu Sharma

Head of department

Dr. Monica Gahlawat

Signature

Date of Submission: _____

Director

Prof. Alok Manke

Acknowledgement

It is our privilege to express our sincere regards to our faculty and project guide, **Prof. Himanshu Sharma**, for their valuable inputs, able guidance, encouragement, whole-hearted cooperation, and constructive criticism throughout the duration of this project.

We deeply express our sincere thanks to our Head of Department **Dr. Monica Gahlawat** and Director **Prof. Alok Manke** for encouraging and allowing us to present the project on **Portify - Automated Portfolio Generator** at our department premises for the partial fulfilment of the requirements leading to the award of **Integrated Master of Computer Application (IMCA)** degree.

We take this opportunity to thank all our lecturers who have directly or indirectly helped with our project. Last but not the least, we express our thanks to our friends for their cooperation and support.

-Purvarajsinh Vaghela

TABLE OF CONTENTS

1. Introduction to the Project.....	1
1.1 Existing System	1
1.2 Need for the New System	2
1.3 Objective of the New System.....	2
1.4 Problem Definition.....	2
1.5 Core Components.....	3
1.6 Project Profile	3
1.7 Assumptions and Constraints.....	4
1.8 Advantages and Limitations of the Proposed System.....	4
2. Requirement Determination & Analysis.....	4
2.1 . Requirement Determination.....	4
2.2 Targeted Users	5
3. System Design	6
3.1 Use Case Diagram.....	6
3.2 Class Diagram.....	8
3.3 Interaction Diagram	15
3.4 Activity Diagram	19
3.5 Data Dictionary	25
4. Development.....	29
4.1 Coding Standards	29
4.2 Screen Shots.....	32
5. Agile Documentation	43
5.1 Agile Project Charter	43
5.2 Agile Roadmap / Schedule.....	44
5.3 Agile Project Plan	44
5.4 Agile User Story	45
5.5 Agile Release Plan	46
5.6 Agile Sprint Backlog.....	47
5.7 Agile Test Plan.....	48
5.8 Earned-value and burn charts.....	50
6. Proposed Enhancements	50
7. Conclusion	51
8. Bibliography	52

1. Introduction

The analysis, design and development details of **Portify (automated portfolio generator)** are presented here in this document. This document will detail the problem to be solved, requirements for solution, system design, and functional operation of the proposed system.

An online portfolio has become a necessity for students, developers, and professionals to display their work, skills, and accomplishments in the digital world today. Unfortunately, creating an online portfolio usually requires knowledge of web development tools such as HTML, CSS, JavaScript, and hosting sites. Many users have encountered issues while trying to develop and maintain a professional portfolio.

With Portify, users have the ability to create a portfolio with pre-built templates, enter their own information, generate professional content (using AI), view a completed portfolio and download instantly through a web-based application.

1.1 Existing System

Today, users build portfolios by:

- ❖ Building and developing HTML / CSS websites from scratch (manual)
- ❖ Using limited function website builders
- ❖ Creating static PDF resumes

The workflow of the current system consists of:

- ❖ A user sketches out their own layout
- ❖ A user types in their HTML and CSS code
- ❖ A user uploads their files to a web host
- ❖ A user will manually update their content (when needed)

Problems with the Current System:

- ❖ Requires technical skills
- ❖ Can be time intensive.
- ❖ Difficult to maintain
- ❖ Limited automation
- ❖ No AI included

1.2 Need for New System

Novice and non-technical users lack adequate methods for making portfolios with current methods available to them. Students and professionals prefer that they can easily create their portfolio without knowledge of web development in the most efficient time possible.

The need for an alternative system arises because of the following:

- ❖ There are no reliable/simple ways to create portfolios as a generator
- ❖ There is too much time spent on designing & developing
- ❖ Websites are complex to maintain/change/update.
- ❖ There is a need to ensure that the content of the portfolio is created by automation.

Portify solves these issues by automating processes, providing templates, and using AI to assist in the creation of portfolios.

1.3 Objective of New System

Portify aims to do the following:

- ❖ Make it easier for users to create a portfolio.
- ❖ Provide users with professionally designed pre-made portfolio templates.
- ❖ Allow users to view their portfolio before they download it.
- ❖ Create a "About Me" page using artificial intelligence.
- ❖ Enable users to easily download their completed portfolio file.
- ❖ Make it easier for administrators to manage their portfolio template and user accounts.

1.4 Problem Definition

Numerous college and professional individuals lack both the technical staff and enough time to develop an online professional portfolio website capable of providing them with an outlet for showcasing their skills. The intent of this project is to develop an online application that will allow a user to create a web-based portfolio with no technical knowledge by using pre-scripted templates and dynamically generated forms.

Through the system, users will be able to sign up for an account, select a pre-made template, input the relevant information regarding their professional experience, view the completed portfolio before creating it, and download all of the generated portfolio documents from the website.

1.5 Core Components

This proposed system will include the following components:

1. Authentication Module
 - ❖ Registration and Login Options
 - ❖ Roles Based Access - Administrator / User
2. Templates Module
 - ❖ Administrator can add a template
 - ❖ Administrator can update or delete a template
 - ❖ User access to view templates
3. Portfolio Creation Module
 - ❖ Dynamic form to input information.
 - ❖ Ability to save portfolio information.
4. AI Content Generation Module
 - ❖ Ability to automatically create an "About Me" section.
5. Preview Module
 - ❖ Ability to view a "live" portfolio.
6. Download Module
 - ❖ Generate Portfolio Files
 - ❖ Ability to Download ZIP
7. Administration Panel
 - ❖ Manage Templates
 - ❖ Manage Users
 - ❖ View Statistics

1.6 Project Profile

Project Name:	Portify – Automated Portfolio Generator
Project Type:	Web Application
Technology Used:	ASP.NET MVC Framework
Database:	SQL Server Management Studio 21
Frontend:	HTML, CSS, JavaScript
Backend:	C#

AI Integration:	Gemini API
-----------------	------------

1.7 Assumptions and Constraints

Assumptions

- ❖ It is assumed that users will have access to the Internet.
- ❖ It is assumed that users will provide valid information.
- ❖ It is assumed that the server and database are available.

Constraints

- ❖ The availability of APIs affects AI features.
- ❖ The number of portfolio templates available will initially be limited.
- ❖ Internet connectivity must be available.

1.8 Advantages and Limitations of The Proposed System

Advantages

- ❖ User-Friendly
- ❖ Time-Efficient
- ❖ No Coding Required
- ❖ Professional Designs
- ❖ AI Generated Content
- ❖ Rapid Downloads & Sharing

Limitations

- ❖ Requires Internet Connection
- ❖ Version 1.0 has Limited Templates
- ❖ AI Content Requires Editing to Your Liking

2. Requirement Determination & Analysis

2.1 Requirement Determination

Functional Requirement

- ❖ User registration and login
- ❖ Template selection
- ❖ Dynamic form generation

- ❖ AI content generation
- ❖ Live preview
- ❖ Portfolio download
- ❖ Admin template management

Non-Functional Requirements:

- ❖ Security
- ❖ Performance
- ❖ Scalability
- ❖ Usability
- ❖ Maintainability

2.2 Targeted Users

1. Students

As one of the main user groups of the system, students are able to:

- ❖ Set up their own portfolio for placement purposes
- ❖ Display the academic projects they have done and internships they have completed
- ❖ Exhibit their technical abilities or soft skills
- ❖ Include certification(s) and other accomplishments
- ❖ Share their portfolio link with recruiters

2. Developers

There are many ways that developers can use the system. They can:

- ❖ Showcase personal and professional projects
- ❖ Exhibit the programming language(s) they know and the technologies they can use
- ❖ Add links to their GitHub or other project(s)
- ❖ Share their portfolio link on their résumé or job application
- ❖ Keep their online profile updated

3. Designers

Designers can also utilize the system by doing the following:

- ❖ Showcase their creative work and samples of their designs
- ❖ Exhibit their UI/UX projects and case studies
- ❖ Present their visual portfolios in a professional manner
- ❖ Share their portfolios with their clients or companies

4. Job Seekers/Professionals

Individuals searching for a job are able to do the following using the system:

- ❖ Create a personal portfolio website relatively quickly
- ❖ Highlight their work history or what skills they possess
- ❖ Share the link to their portfolio with their interviewer in an interview
- ❖ Keep an online professional presence

5. Freelancers

Freelancers can apply the system by doing the following:

- ❖ Showcase their previous client projects
- ❖ Present their services and skills
- ❖ Share their portfolio link with potential clients
- ❖ Build their credibility through an online presence

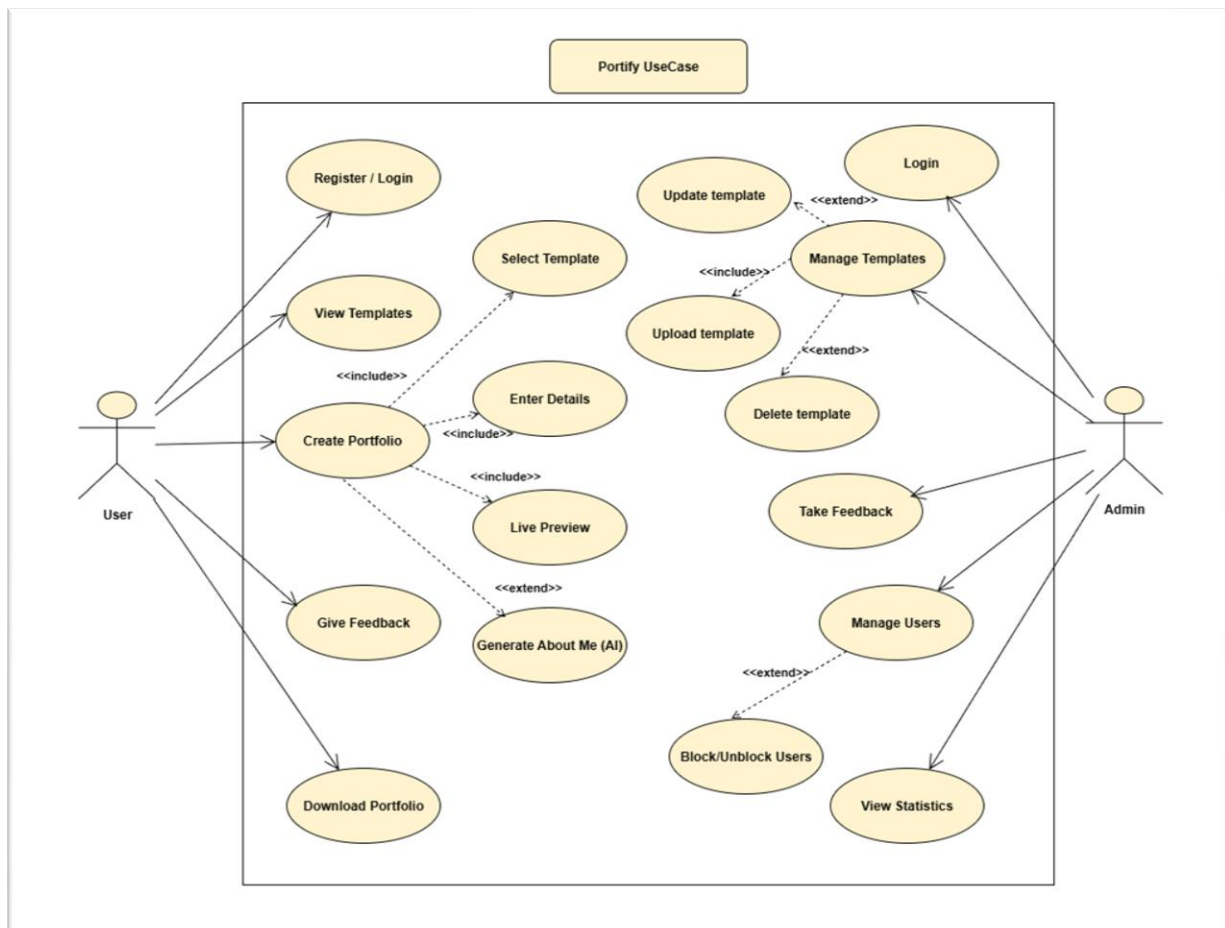
6. Admin

The admin is responsible for managing the system. The admin is able to do:

- ❖ Manage the templates (e.g., adding, editing, deleting)
- ❖ Manage the users of the system
- ❖ Monitor how the system is used and track the statistics
- ❖ Maintain the data and functionality of the system.

3. System Design

3.1 Use Case Diagram (Explanation)



The Use Case Diagram represents the interaction between users and the Portify system. It shows the different functionalities available to each actor and how they interact with the system.

Actors in the System

There are two types of actors who can interact with the system: the user and the admin. Users are people who access the system to create and edit their own portfolios, while admins are responsible for maintaining the system and helping users with issues related to their portfolios.

1. Users will:

- ❖ Register for an account
- ❖ Login to their account
- ❖ Browse through available templates
- ❖ Select a template(s) to use in their portfolio
- ❖ Provide information about themselves and their projects
- ❖ Use AI to create an "About Me" page
- ❖ View their portfolio in real-time
- ❖ Download their completed portfolio
- ❖ Provide feedback about the system and templates

The two actions of Providing Information and Viewing Portfolio in Real Time are necessary to the Portfolio Creation use case.

2. Admin will:

- ❖ Login to their account
- ❖ Upload/update/delete templates
- ❖ Create/manage users
- ❖ Block/unblock users if needed
- ❖ View statistics for the system
- ❖ Read feedback left by users

The actions of Upload Template, Update Template, and Delete Template are extensions of the Manage Template use case.

System Boundary

The rectangle in the diagram represents the **Portify System**, which contains all functionalities such as portfolio creation, template management, feedback handling, and statistics monitoring.

Relationships Used

Include Relationship (<<include>>)

Used for functions that must be part of other functions.

Example:

- ❖ Create Portfolio includes Enter Details and Live Preview.

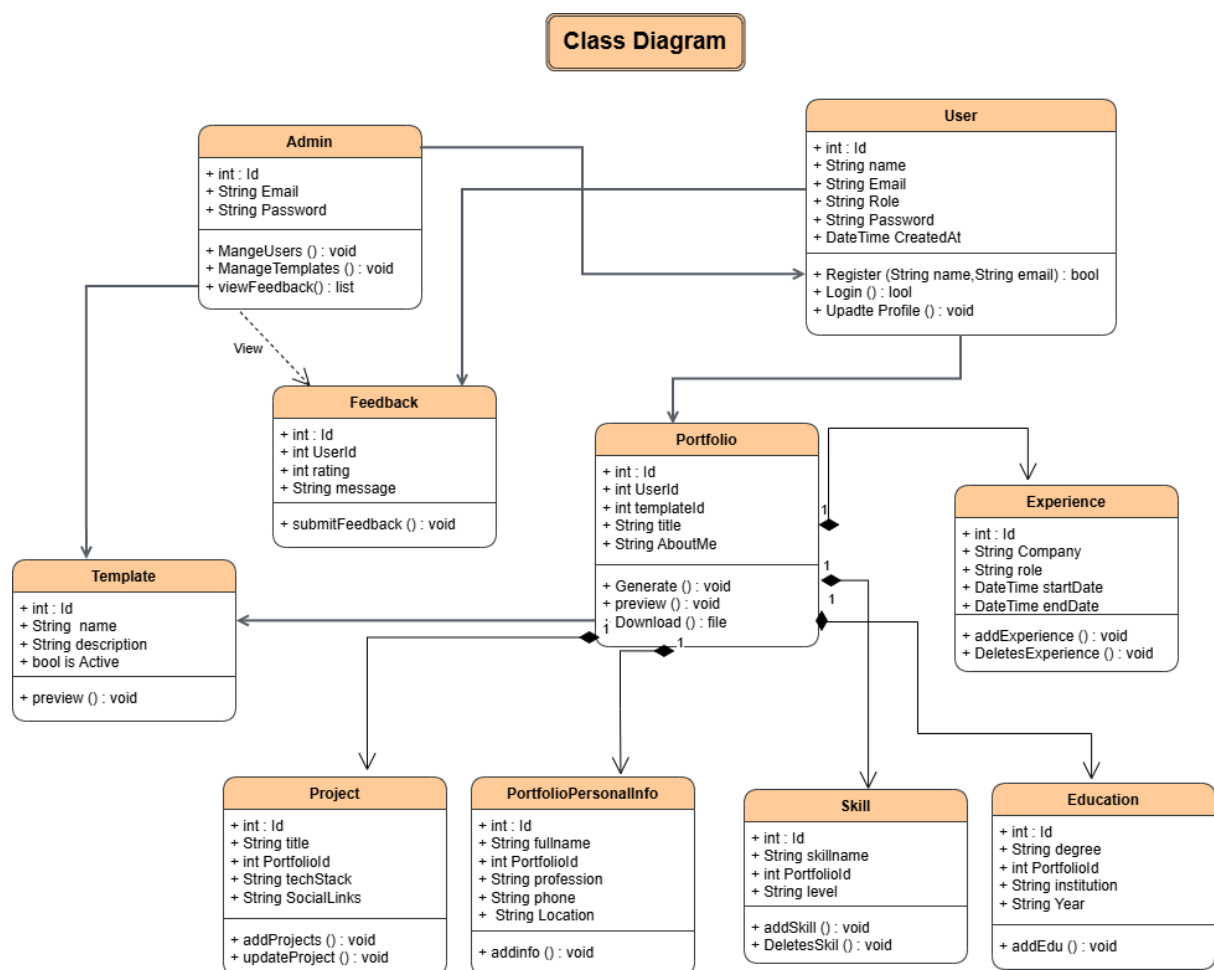
Extend Relationship (<<extend>>)

Used where a function is optional or an extension of another function.

Example:

- ❖ Generate About Me (AI) extends portfolio creation.
- ❖ Upload/Update/Delete Template extend Manage Templates.

3.2 Class Diagram (Explanation)



The Class Diagram represents the internal structure of the Portify system. It shows the different classes, their attributes, methods, and the relationships between them. The diagram illustrates how data flows within the system and how different components interact to build a complete portfolio.

Classes in the System

1. Admin Class

The admin class represents the administrator of the system who manages users, templates, and feedback.

Attributes:

- ❖ Id
- ❖ Email
- ❖ Password

Methods:

- ❖ ManageUsers()
- ❖ ManageTemplates()
- ❖ viewFeedback()

The admin has full control over the system and can monitor activities and maintain data.

2. User Class

The User class represents individuals who create and manage their portfolios.

Attributes:

- ❖ Id
- ❖ name
- ❖ Email
- ❖ Role
- ❖ Password
- ❖ CreatedAt

Methods:

- ❖ Register(String name, String email)
- ❖ Login()
- ❖ Update Profile()

Users can register, log in, and update their profiles before creating portfolios.

3. Portfolio Class

The Portfolio class is the central class of the system where all user information is stored and managed.

Attributes:

- ❖ Id
- ❖ UserId
- ❖ templateId
- ❖ title
- ❖ AboutMe

Methods:

- ❖ Generate()
- ❖ preview()
- ❖ Download()

A portfolio is created by a user using a selected template and contains all personal and professional details.

4. Template Class

The Template class stores the predefined designs available for portfolio creation.

Attributes:

- ❖ Id
- ❖ name
- ❖ description
- ❖ isActive

Methods:

- ❖ preview()

Templates are managed by the admin and used by users to design their portfolios.

5. Feedback Class

The Feedback class stores feedback provided by users.

Attributes:

- ❖ Id
- ❖ UserId
- ❖ rating
- ❖ message

Methods:

- ❖ submitFeedback()

Users submit feedback, and admins can view it.

6. Project Class

The Project class represents projects added to a portfolio.

Attributes:

- ❖ Id
- ❖ title
- ❖ PortfolioId
- ❖ techStack
- ❖ SocialLinks

Methods:

- ❖ addProjects()
- ❖ updateProject()

Each project belongs to a specific portfolio.

7. PortfolioPersonalInfo Class

This class stores personal details of the user within the portfolio.

Attributes:

- ❖ Id
- ❖ fullname
- ❖ PortfolioId
- ❖ profession
- ❖ phone
- ❖ Location

Methods:

- ❖ addinfo()

It contains personal and contact information of the user.

8. Skill Class

The Skill class stores the skills added to a portfolio.

Attributes:

- ❖ Id

- ❖ skillname
- ❖ PortfolioId
- ❖ level

Methods:

- ❖ addSkill()
- ❖ DeleteSkill()

Users can add and remove skills from their portfolio.

9. Experience Class

The Experience class represents work experience details.

Attributes:

- ❖ Id
- ❖ Company
- ❖ role
- ❖ startDate
- ❖ endDate

Methods:

- ❖ addExperience()
- ❖ DeleteExperience()

It stores professional experience of the user.

10. Education Class

The Education class stores educational details.

Attributes:

- ❖ Id
- ❖ degree
- ❖ PortfolioId
- ❖ institution
- ❖ Year

Methods:

- ❖ addEdu()

It contains academic background information.

Relationships and Arrow Types Used in Class Diagram

The class diagram uses different types of relationships to represent how classes are connected and interact with each other. These relationships are shown using various arrows and symbols.

1. Association (→)

Association represents a general relationship between two classes where one class uses or is connected to another.

Representation:

A simple straight line with an arrow.

Examples from Diagram:

- ❖ Admin → User (Admin manages users)
- ❖ Admin → Template (Admin manages templates)
- ❖ User → Portfolio (User creates a portfolio)

This indicates that one class is related to another but does not strongly depend on it.

2. One-to-Many Relationship

This shows that one object of a class is connected to multiple objects of another class.

Examples:

- ❖ One Portfolio → Many Projects
- ❖ One Portfolio → Many Skills
- ❖ One Portfolio → Many Education records

This means a single portfolio can contain multiple entries of these types.

3. Composition (◆)

Composition is a strong relationship where one class completely owns another class. If the parent class is deleted, the child classes are also deleted.

Representation:

A filled diamond (◆) near the parent class.

Examples from Diagram:

- ❖ Portfolio ◆ Project
- ❖ Portfolio ◆ Skill
- ❖ Portfolio ◆ Education
- ❖ Portfolio ◆ PortfolioPersonalInfo

This means these classes cannot exist independently without the Portfolio.

4. Dependency (Dashed Arrow)

Dependency represents a weak relationship where one class depends on another temporarily.

Representation:

A dashed arrow.

Example from Diagram:

❖ **Admin → Feedback (View)**

This indicates that Admin uses Feedback only for viewing purposes and does not own it.

5. Direction of Arrows

The direction of the arrow indicates the flow of control or usage.

❖ **If the arrow points from Admin to User → Admin controls or manages User**

❖ **If the arrow points from User to Portfolio → User creates or uses Portfolio**

Conclusion

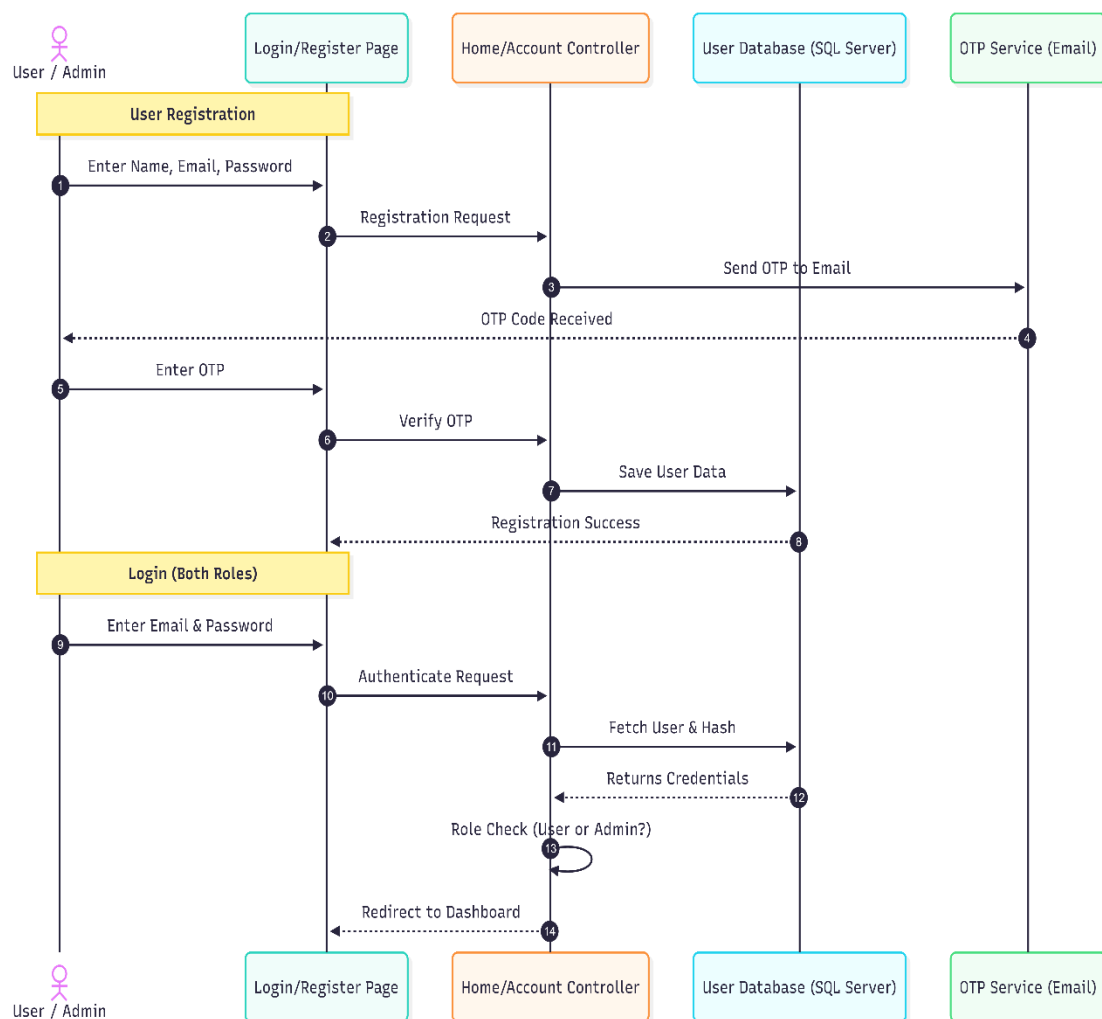
Different arrow types in the class diagram help to clearly define the relationships between classes. Association shows general connections, composition shows strong ownership, dependency shows temporary usage, and one-to-many relationships define how multiple objects are related. These relationships make the system design clearer and more understandable.

3.4 Intersection Diagram (Explanation)

The Intersection Diagram represents how different system processes interact with each other at specific points. It focuses on the flow of control between multiple actors and system components and shows how different operations are connected in real-time.

Unlike activity diagrams that show step-by-step flow, intersection diagrams highlight where different processes meet, interact, and exchange information within the system.

Intersection – User Register/Login and Admin Login



This intersection shows how both User and Admin authentication processes interact with the system.

- ❖ The process begins when a User or Admin enters login or registration details.
- ❖ The system validates the credentials using the database.

- ❖ If the user is new, the registration process stores data in the system.
- ❖ If the user already exists, the system performs login authentication.
- ❖ After successful verification:
 - ❖ Users are redirected to the portfolio dashboard.
 - ❖ Admins are redirected to the admin panel.

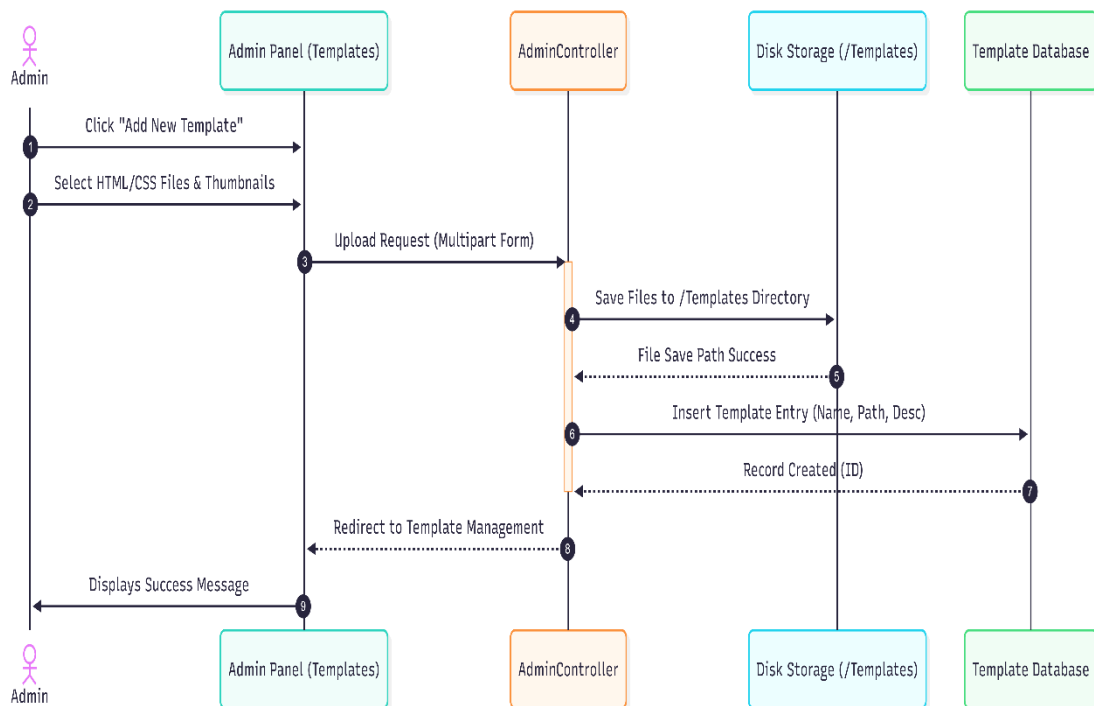
Intersection Point:

The common intersection occurs at the authentication module, where both User and Admin requests are validated and processed.

Relationship:

- ❖ Both actors share a common validation process.
- ❖ The system decides the next flow based on the role (User/Admin).

Intersection – Admin Template Upload



This intersection illustrates how the Admin interacts with the Template module.

- ❖ Admin logs into the system.
- ❖ Admin selects the option to add/upload a template.
- ❖ The system accepts template data (HTML, CSS, preview, config files).
- ❖ The template is stored in the database.
- ❖ The system updates the template list for users.

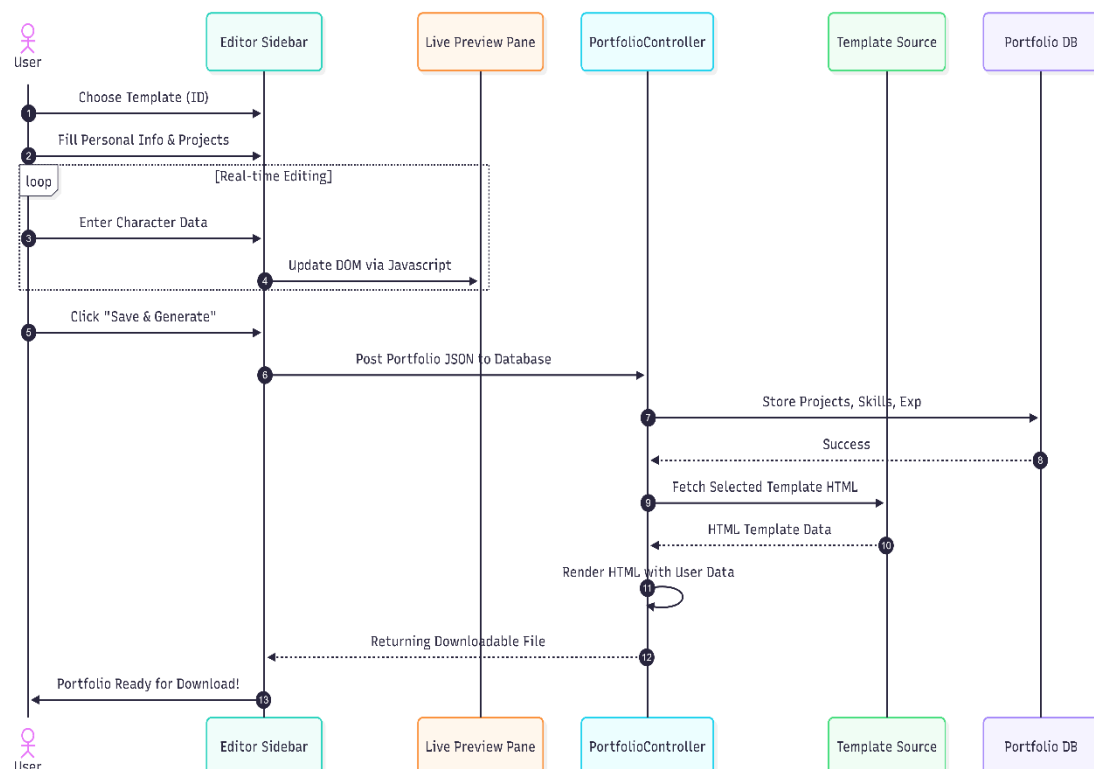
Intersection Point:

The interaction between Admin actions and Template storage system.

Relationship:

- ❖ Admin → Template (management relationship)
- ❖ System validates and stores template data before making it available.

Intersection – User Creates Portfolio



This intersection shows how multiple modules interact when a User creates a portfolio.

- ❖ User logs into the system.
- ❖ User selects a template.
- ❖ User enters personal, educational, and project details.
- ❖ The system may generate AI-based “About Me” content.
- ❖ The portfolio is generated and stored.
- ❖ User can preview and download the portfolio.

Intersection Points:

- ❖ User ↔ Template selection
- ❖ User ↔ Portfolio creation module
- ❖ Portfolio ↔ AI content generation
- ❖ Portfolio ↔ Preview & Download module

Relationships:

- ❖ User interacts with multiple system components simultaneously.
- ❖ Portfolio acts as a central entity connecting all modules.
- ❖ Data flows between input forms, AI module, and output generation.

Relationships:

The Intersection Diagram helps in understanding how different parts of the system work together at common points. It clearly shows:

- ❖ Interaction between actors and system modules
- ❖ Flow of data across multiple components
- ❖ Real-time coordination between processes

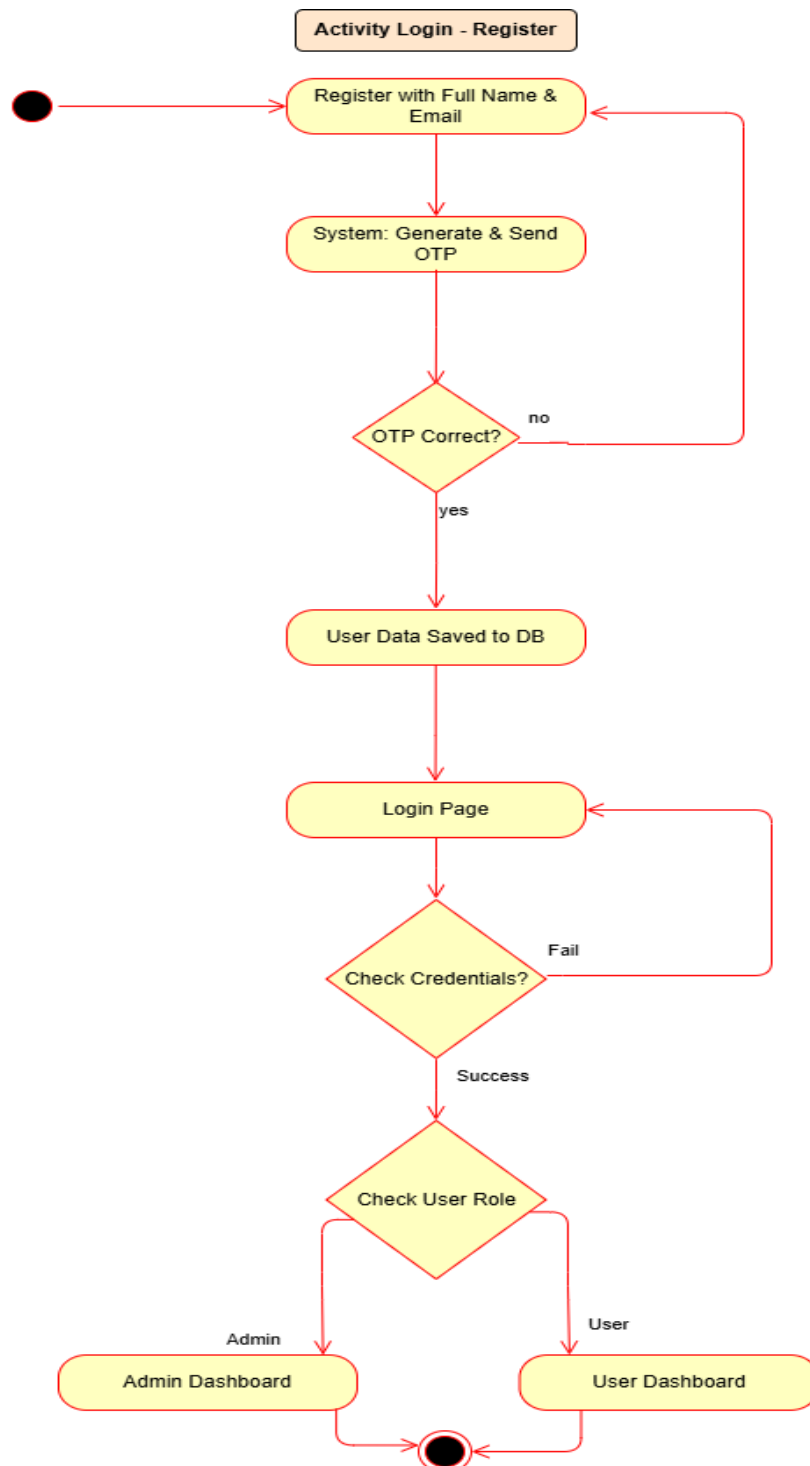
These intersections ensure that the system functions smoothly by connecting authentication, template management, and portfolio generation into a unified workflow.

3.4 Activity Diagram (Explanation)

The Activity Diagram represents the workflow of the Portify system by showing the sequence of activities performed by users and admins. It illustrates how different actions are connected, how decisions are made, and how the system flows from one step to another.

It helps in understanding the dynamic behaviour of the system and the step-by-step process involved in portfolio creation, user authentication, and admin operations.

Activity – Login / Register



This activity diagram describes the process of user authentication in the system.

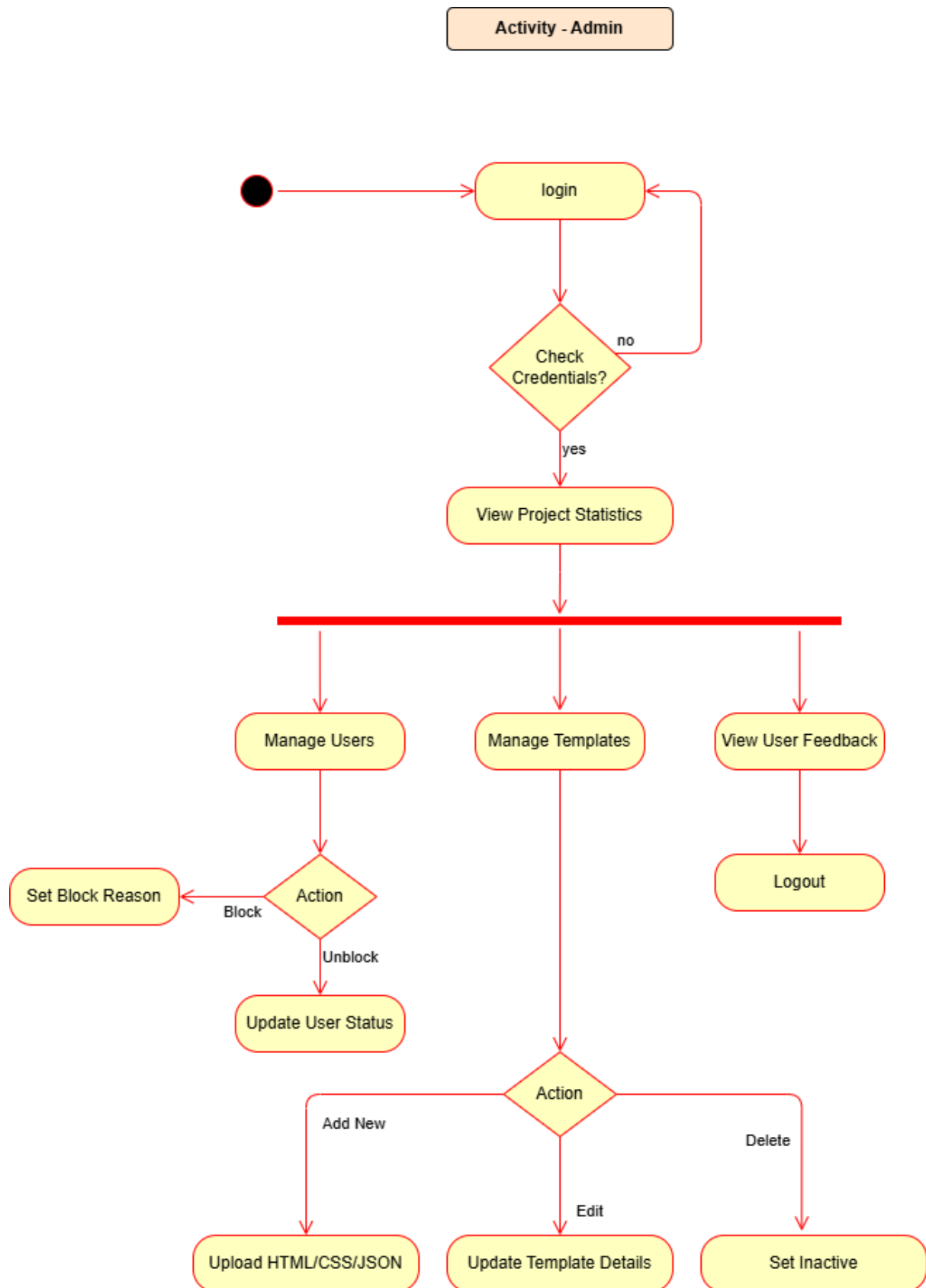
The process begins when a user accesses the system. The user is given two options: Register or Login.

- ❖ If the user selects Register, they must enter details such as name, email, and password.
- ❖ The system validates the input data.
 - ❖ If the data is valid, the account is created successfully, and the user is redirected to the login page.
 - ❖ If invalid, an error message is displayed, and the user is asked to re-enter the details.
- ❖ If the user selects Login, they enter their email and password.
- ❖ The system verifies the credentials:
 - ❖ If correct, the user is successfully logged in and redirected to the dashboard.
 - ❖ If incorrect, an error message is shown.

Relationships Used:

- ❖ Sequential Flow (→): Represents the step-by-step process (Enter Details → Validate → Success/Failure).
- ❖ Decision Node (◇): Used to check whether user input or credentials are valid.
- ❖ Merge Node: Combines multiple paths (Register/Login) into a single flow after successful authentication.

Activity – Admin



This activity diagram explains the workflow for admin operations.

The process begins when the admin logs into the system.

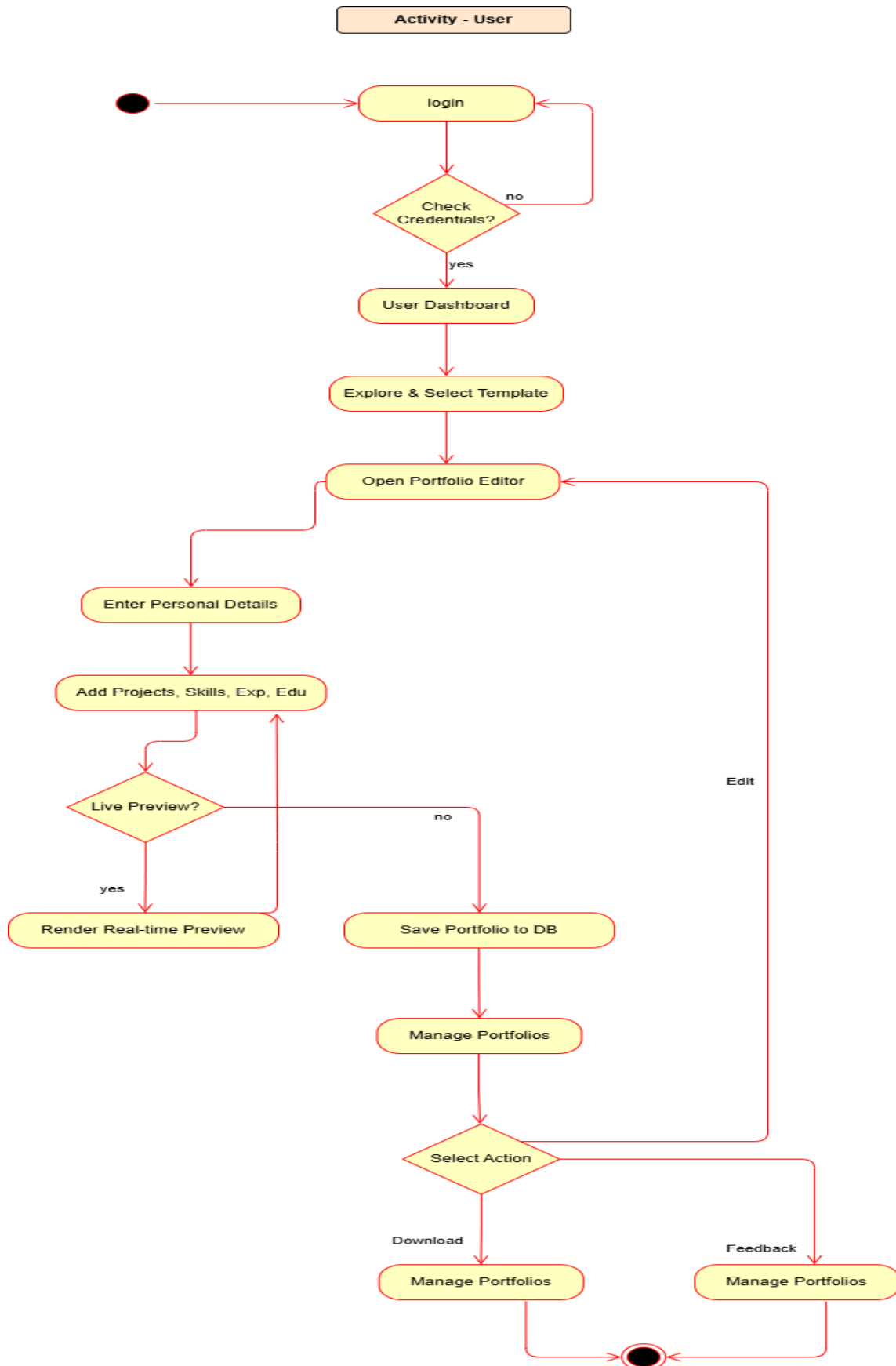
- ❖ The admin accesses the dashboard.
- ❖ The admin can perform multiple operations such as:
 - ❖ Manage users (add, block, unblock)
 - ❖ Manage templates (upload, update, delete)
 - ❖ View system statistics
 - ❖ Review user feedback

Each operation follows its own flow but originates from the admin dashboard.

Relationships Used:

- ❖ **Sequential Flow (→):** Represents the process flow from login to dashboard and actions.
- ❖ **Decision Node (◇):** Used to select which operation the admin wants to perform.
- ❖ **Fork Node:** Allows admin to perform different management tasks independently.
- ❖ **Merge Node:** Brings different operations back to the main dashboard flow.

Activity – User



The activity diagrams collectively show how the system operates from start to end.

- ❖ The system starts with authentication (Login/Register).
- ❖ Based on the role (User/Admin), the flow diverges:
 - ❖ **User Flow:** Portfolio creation, preview, and download
 - ❖ **Admin Flow:** System management and monitoring

Relationships Used in Overall Diagram:

- ❖ **Start Node (●):** Indicates the beginning of the process.
- ❖ **End Node (◎):** Represents completion of the process.
- ❖ **Swimlanes (if used):** Separate responsibilities between User and Admin.
- ❖ **Control Flow Arrows (→):** Show direction of execution.

Conclusion:

The Activity Diagram provides a clear visualization of how tasks are performed within the Portify system. It highlights the flow of control, decision-making points, and interactions between different components. By using elements like decision nodes, loops, and parallel flows, the diagram ensures that the system behaviour is easy to understand and logically structured.

3.5 Data Dictionary

This section contains the data dictionary for the Portify project, providing detailed information about the database structure, table definitions, and field constraints.

Table Name: Users

Purpose: Stores information about users (Admins and regular Users).

Field Name	Data Type	Constraints	Description
Id	INT	PRIMARY KEY, IDENTITY	Unique identifier for each user.
FullName	NVARCHAR(100)	NOT NULL	The user's full name.
Email	NVARCHAR(100)	NOT NULL, UNIQUE	User's email address used for login.
PasswordHash	NVARCHAR(255)	NOT NULL	Hashed version of the user's password.
Role	NVARCHAR(20)	NOT NULL, CHECK (Admin/User)	Defines user access level (Admin or User).
IsBlocked	BIT	DEFAULT(0)	Indicates if the user is blocked from the system.
CreatedAt	DATETIME	DEFAULT(GETDATE())	The date and time when the user registered.

Table Name: Templates

Purpose: Stores metadata for available portfolio templates.

Field Name	Data Type	Constraints	Description
Id	INT	PRIMARY KEY, IDENTITY	Unique identifier for the template.
TemplateName	NVARCHAR(100)	NOT NULL	Name of the template design.
Description	NVARCHAR(255)	NULL	Brief description of the template's style.
ThumbnailPath	NVARCHAR(255)	NULL	Path to the preview image for the template.
HtmlPath	NVARCHAR(255)	NOT NULL	Path to the base HTML file for generation.
CssPath	NVARCHAR(255)	NOT NULL	Path to the associated CSS file.

JsPath	NVARCHAR(255)	NULL	Path to the associated JavaScript file.
ConfigPath	NVARCHAR(255)	NOT NULL	Path to the template configuration file.
IsActive	BIT	DEFAULT(1)	Flag to show/hide the template for selection.
CreatedAt	DATETIME	DEFAULT(GETDATE())	Date the template was added.

Table Name: Portfolios

Purpose: Stores the core portfolio instances created by users.

Field Name	Data Type	Constraints	Description
Id	INT	PRIMARY KEY, IDENTITY	Unique identifier for the portfolio.
UserId	INT	FOREIGN KEY (Users.Id)	ID of the user who owns the portfolio.
TemplateId	INT	FOREIGN KEY (Templates.Id)	ID of the selected template.
Title	NVARCHAR(150)	NULL	User-defined title for the portfolio.
AboutMe	NVARCHAR(MAX)	NULL	"About Me" section content.
CreatedAt	DATETIME	DEFAULT(GETDATE())	Date when the portfolio was generated.

Table Name: PortfolioPersonalInfo

Purpose: Stores personal details specific to a generated portfolio.

Field Name	Data Type	Constraints	Description
Id	INT	PRIMARY KEY, IDENTITY	Unique identifier for the entry.
PortfolioId	INT	FOREIGN KEY (Portfolios.Id)	Links info to a specific portfolio.
FullName	NVARCHAR(100)	NULL	Name to display on the portfolio.
Profession	NVARCHAR(100)	NULL	Professional title (e.g., Full Stack Dev).
Email	NVARCHAR(100)	NULL	Contact email for the portfolio visitor.
Phone	NVARCHAR(20)	NULL	Contact phone number.
Location	NVARCHAR(100)	NULL	User's location (City, Country).

ProfileImagePath	NVARCHAR(255)	NULL	Path to the uploaded profile picture.
-------------------------	---------------	------	---------------------------------------

Table Name: Experiences

Purpose: Stores work experience entries for resumes/portfolios.

Field Name	Data Type	Constraints	Description
Id	INT	PRIMARY KEY, IDENTITY	Unique identifier for the entry.
PortfolioId	INT	FOREIGN KEY (Portfolios.Id)	Links experience to a portfolio.
CompanyName	NVARCHAR(150)	NULL	Name of the employer/company.
Role	NVARCHAR(100)	NULL	Job title/position held.
StartDate	DATE	NULL	Date the role began.
EndDate	DATE	NULL	Date the role ended (null if current).
Description	NVARCHAR(MAX)	NULL	Description of responsibilities.

Table Name: Projects

Purpose: Stores project details to be showcased in the portfolio.

Field Name	Data Type	Constraints	Description
Id	INT	PRIMARY KEY, IDENTITY	Unique identifier for the project.
PortfolioId	INT	FOREIGN KEY (Portfolios.Id)	Links project to a portfolio.
ProjectTitle	NVARCHAR(150)	NULL	Name of the project.
Description	NVARCHAR(MAX)	NULL	Overview of the project work.
TechStack	NVARCHAR(255)	NULL	Technologies used (e.g., React, SQL).
GitHubLink	NVARCHAR(255)	NULL	URL to the source code repository.
LiveLink	NVARCHAR(255)	NULL	URL to the hosted project demo.

Table Name: Skills

Purpose: Stores technical or soft skills and their proficiency levels.

Field Name	Data Type	Constraints	Description
Id	INT	PRIMARY KEY, IDENTITY	Unique identifier for the entry.
PortfolioId	INT	FOREIGN KEY (Portfolios.Id)	Links skill to a portfolio.
SkillName	NVARCHAR(100)	NULL	Name of the skill (e.g., Java).
SkillLevel	NVARCHAR(50)	NULL	Level (e.g., Beginner, Advanced).

Table Name: SocialLinks

Purpose: Stores social media URLs for the portfolio.

Field Name	Data Type	Constraints	Description
Id	INT	PRIMARY KEY, IDENTITY	Unique identifier for the link.
PortfolioId	INT	FOREIGN KEY (Portfolios.Id)	Links social link to a portfolio.
Platform	NVARCHAR(50)	NULL	e.g. LinkedIn, Twitter, GitHub.
Url	NVARCHAR(255)	NULL	Link to the user's profile.

Table Name: Feedback

Purpose: Stores user ratings and feedback for the platform itself.

Field Name	Data Type	Constraints	Description
Id	INT	PRIMARY KEY, IDENTITY	Unique identifier for the feedback.
UserId	INT	FOREIGN KEY (Users.Id)	The user giving the feedback.
PortfolioId	INT	FOREIGN KEY (Portfolios.Id)	Optional reference to a portfolio.
Rating	INT	CHECK (1-5)	Star rating given by the user.
Message	NVARCHAR(MAX)	NULL	Review comment or message.
CreatedAt	DATETIME	DEFAULT(GETDATE())	Date the feedback was submitted.

Table Name: Downloads

Purpose: Tracks portfolio export/download history.

Field Name	Data Type	Constraints	Description
Id	INT	PRIMARY KEY, IDENTITY	Unique identifier for the record.
PortfolioId	INT	FOREIGN KEY (Portfolios.Id)	The portfolio that was downloaded.
DownloadedAt	DATETIME	DEFAULT(GETDATE())	Timestamp of the download action.

Table Name: Education

Purpose: Stores academic qualifications and education history.

Field Name	Data Type	Constraints	Description
Id	INT	PRIMARY KEY, IDENTITY	Unique identifier for the entry.
PortfolioId	INT	FOREIGN KEY (Portfolios.Id)	Links education to a portfolio.
Degree	NVARCHAR(150)	NULL	Name of the degree or course.
Institution	NVARCHAR(150)	NULL	Name of the school, college, or university.
Year	NVARCHAR(50)	NULL	Passing year or duration.

4 Development

The development of Portify was conducted using an agile, solo-developer approach, where the entirety of the project lifecycle—from initial architectural design to final deployment—was managed by a single member. Work distribution was handled through a task-based 'separation of concerns' strategy, partitioning the project into distinct development phases: Database Schema design, Backend logic implementation using C# ASP.NET, and Frontend UI/UX development. The process of integration was iterative; each module was independently developed and unit-tested before being integrated into the core MVC framework. This systematic integration ensured that the automated portfolio generation logic remained decoupled from the presentation layer, allowing for seamless updates and high system stability throughout the development cycle.

4.1 Coding Standards

To maintain a clean, readable, and maintainable codebase, Portify adheres to industry-standard naming conventions and structural practices. The following standards were consistently applied across the project:

4.1.1 C# / Backend Naming Conventions

The backend logic follows standard Microsoft C# coding conventions to ensure consistency within the .NET environment:

Classes and Structs:

Named using PascalCase naming convention.

Examples: PortfolioController, User, PortifyDbContext

Methods:

Named using PascalCase and generally begin with a verb to clearly describe the action being performed.

Examples: GetTemplateById, SavePortfolio, RenderTemplate

Properties:

Public properties are named using PascalCase for consistency and readability.

Examples: FullName, Email, CreatedAt

Local Variables:

Named using camelCase to differentiate them from classes, methods, and properties.

Examples: templateId, portfolioTitle, jsonData

Namespaces:

Follow the project folder hierarchy and use the PascalCase naming convention.

Examples: Portify.Models, Portify.Controllers

4.1.2 Frontend (JavaScript & CSS) Naming Conventions

For the presentation layer, the following conventions are used to ensure cross-browser compatibility and ease of debugging:

JavaScript Variables & Functions:

Use camelCase naming convention for variables and functions.

Examples: sendMessage, typingIndicatorHTML, closeBtn

CSS Classes & IDs:

Use kebab-case naming convention (lowercase words separated with hyphens) for better readability and consistency in HTML and CSS files.

Examples: .cb-msg-group, .project-card, #chatbot-fab

Constants:

Global constants are generally written using the UPPERCASE_SNAKE_CASE naming convention where applicable.

4.1.3 Frontend (JavaScript & CSS) Naming Conventions

The database schema follows a structured approach to simplify Object-Relational Mapping (ORM):

Tables:

Named using PascalCase naming convention and are pluralized to represent collections of data.

Examples: Users, Portfolios, SocialLinks

Columns:

Named using PascalCase for consistency and readability.

Examples: UserId, Profession, IsActive

Primary Keys:

Primary key fields are consistently named Id across all tables to maintain uniformity throughout the database structure.

4.1.4 Code Formatting and Style

The database schema follows a structured approach to simplify Object-Relational Mapping (ORM):

Indentation:

Standard 4-space indentation is used throughout all code files to maintain consistency and improve readability.

Braces:

The Allman brace style is followed, where opening and closing braces are placed on separate lines to improve vertical readability and code structure.

Comments:

XML Documentation:

Complex methods use `///` tags to provide proper documentation, tooltips, and better code understanding.

Inline Comments:

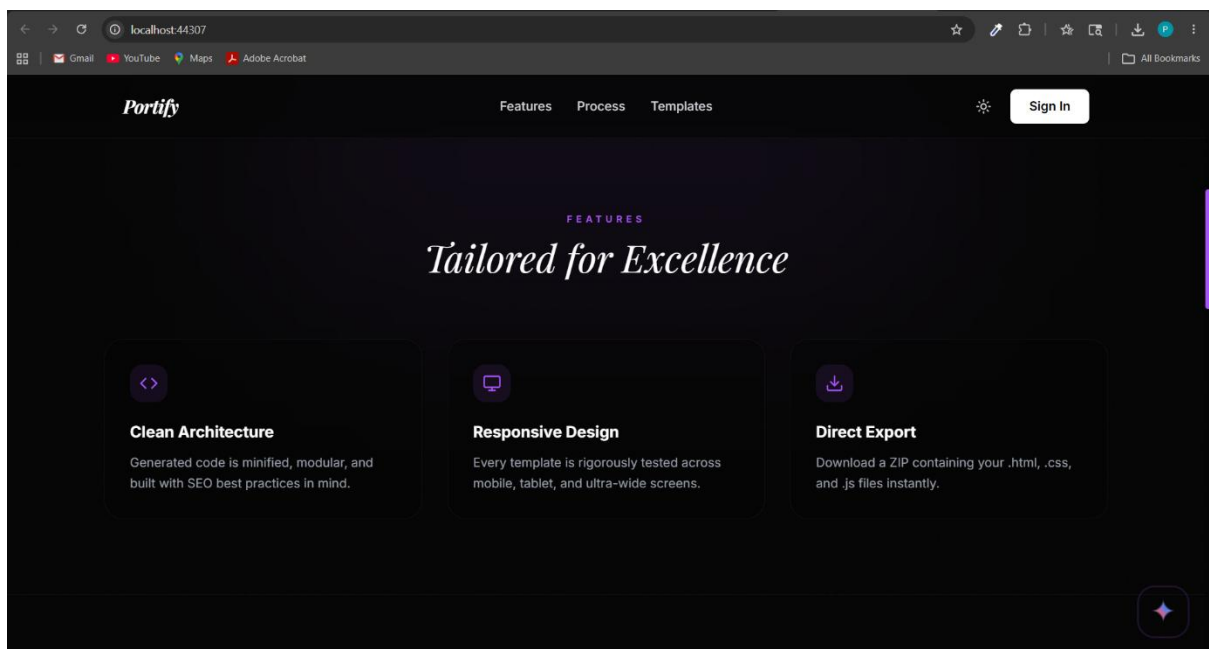
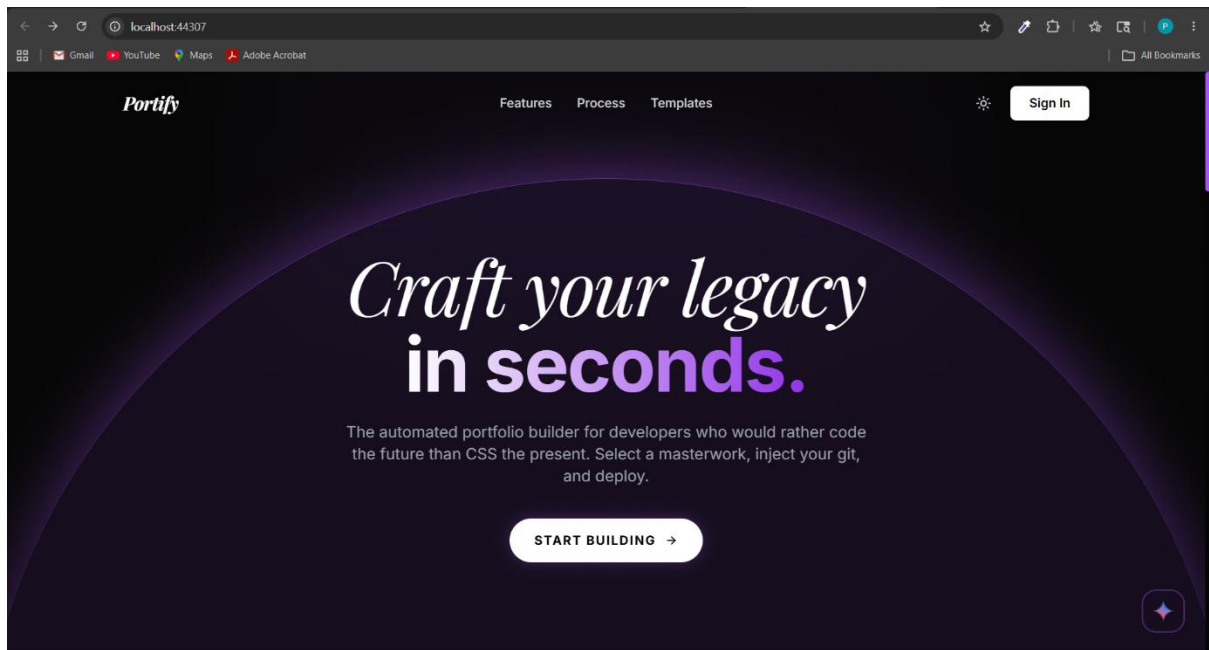
Inline comments are used only where necessary to explain why a particular logic or implementation is used rather than describing what the code is doing.

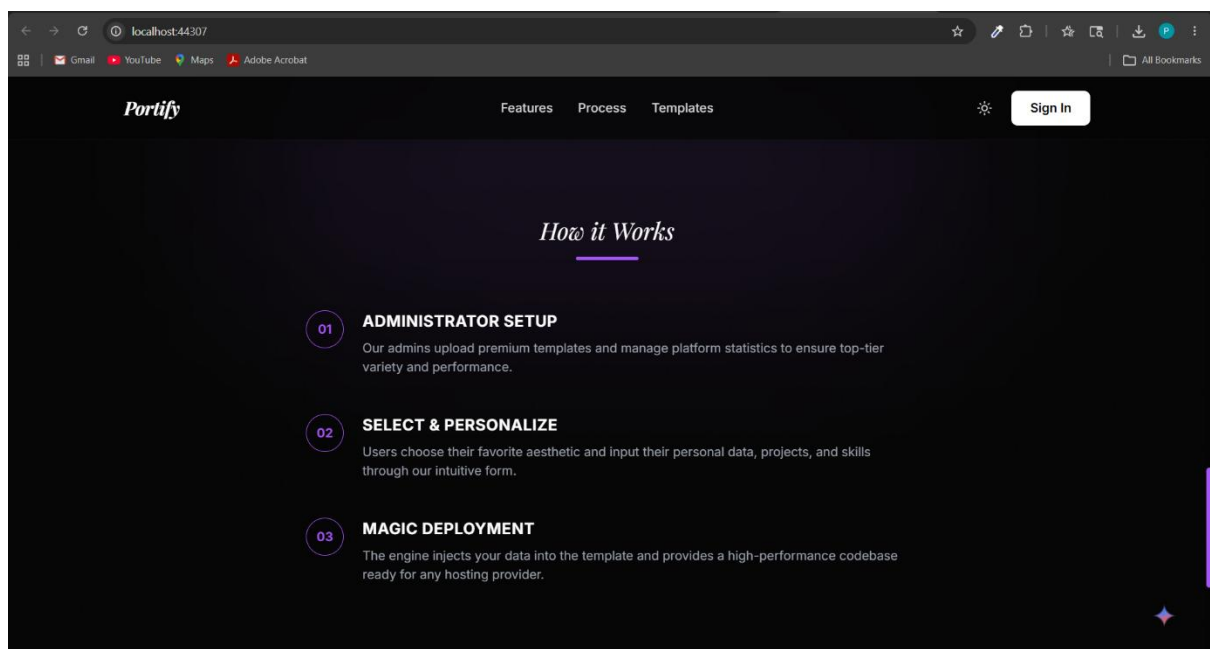
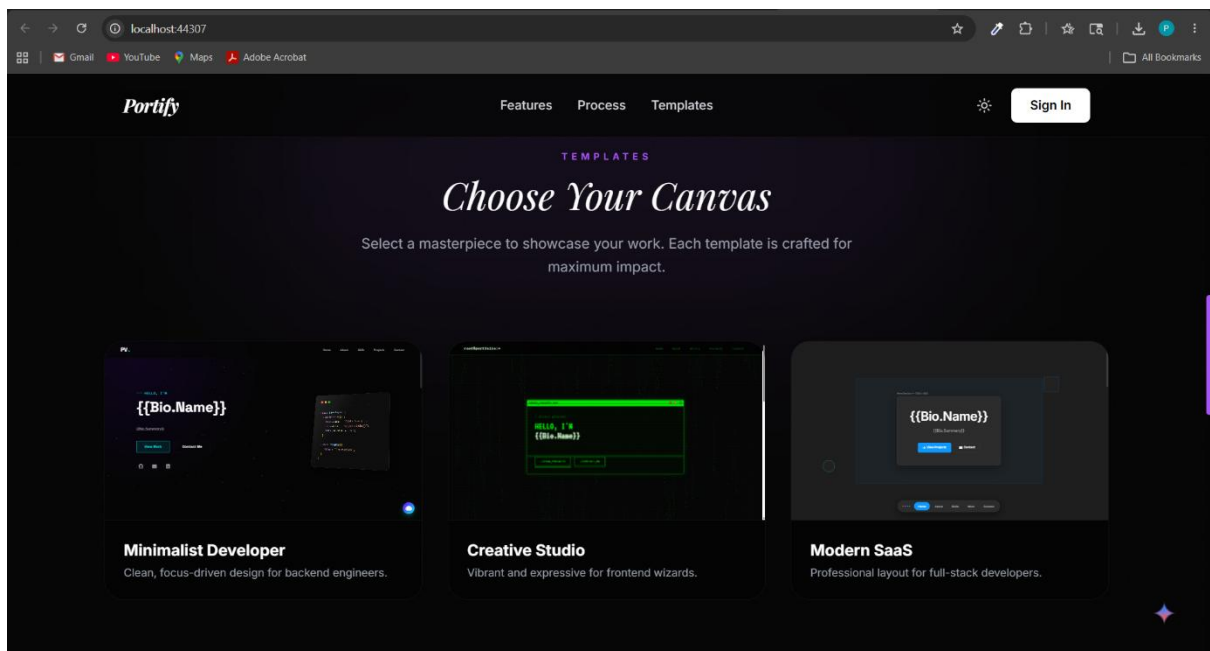
File Organization:

The project follows the MVC (Model-View-Controller) architecture pattern, ensuring a clear separation between data handling, business logic, and user interface components.

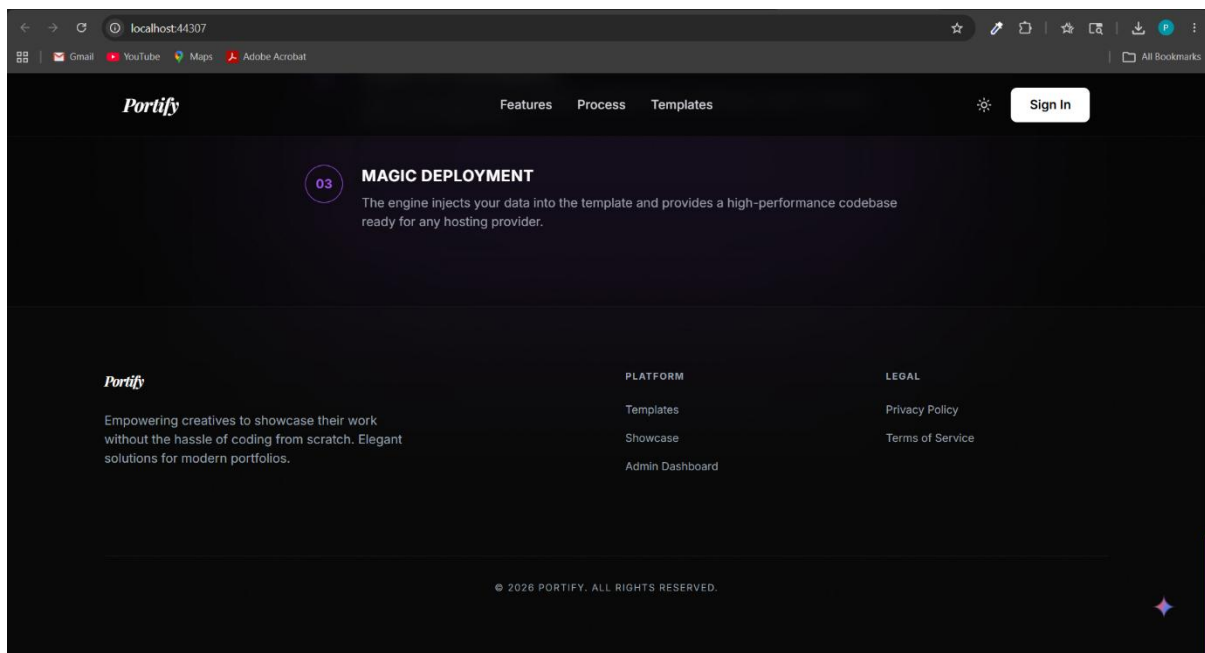
4.2 Screenshots

4.2.1 Home Page

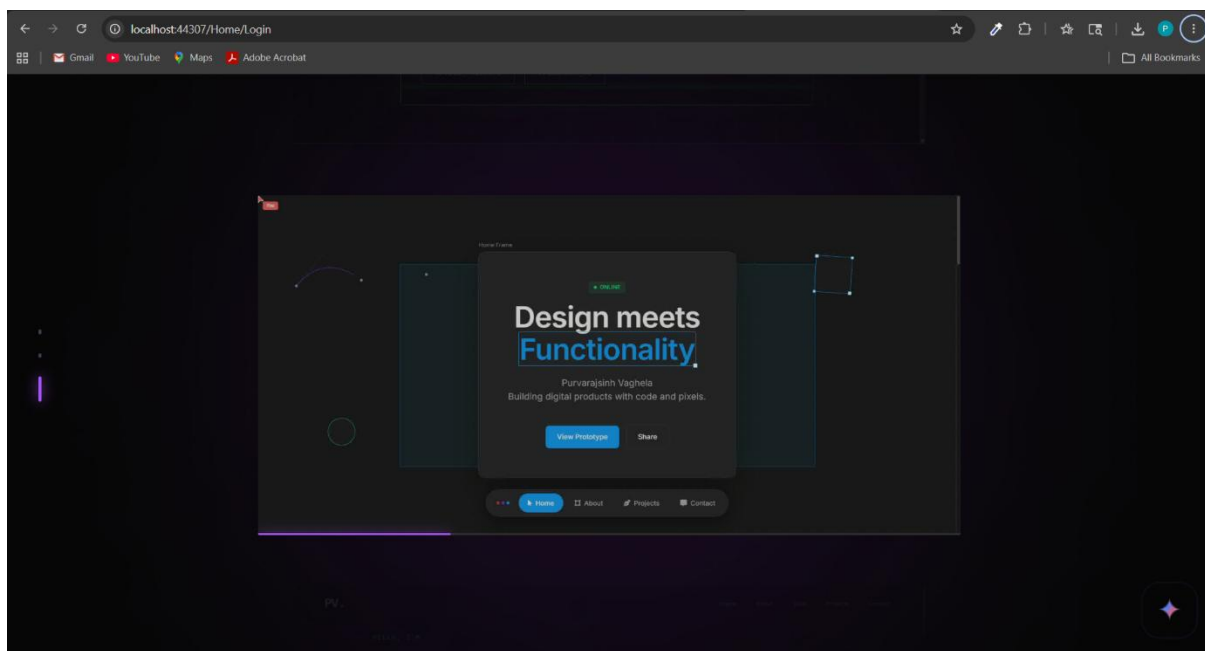




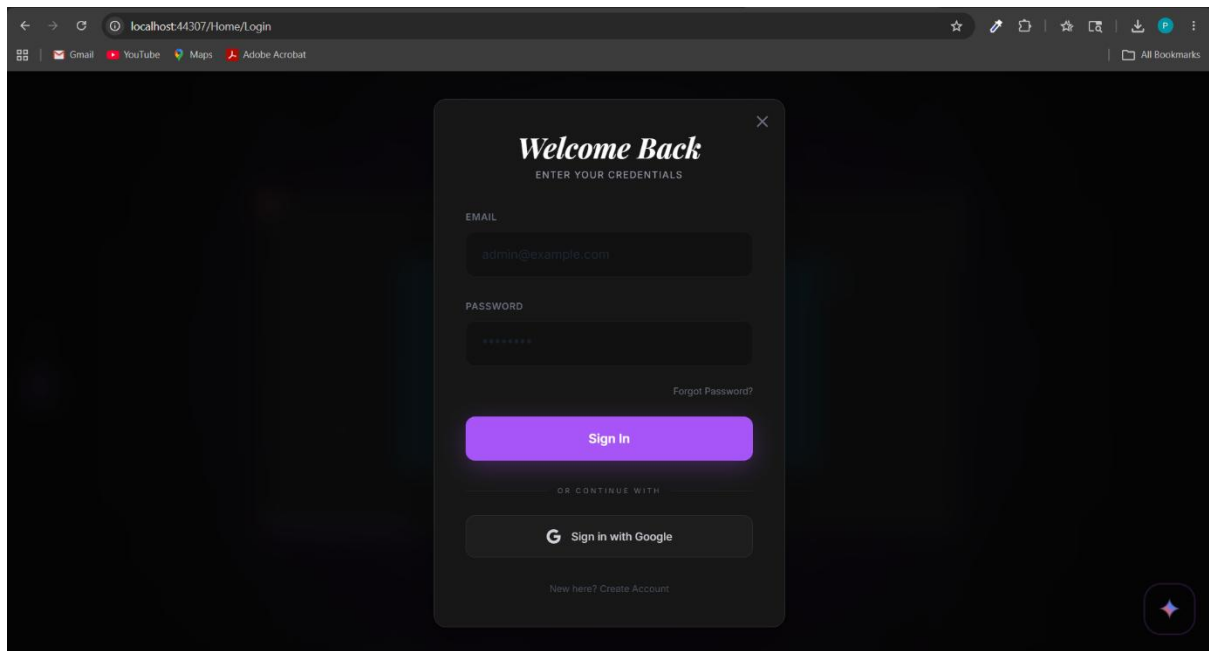
Portify – Automated Portfolio Generator



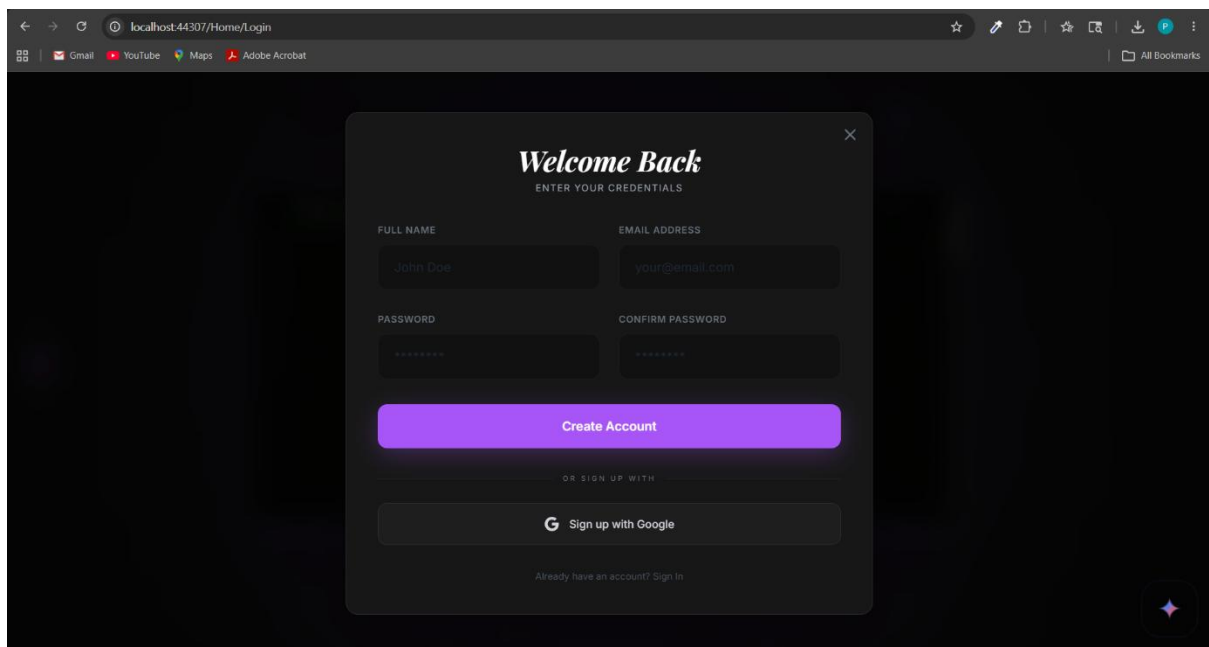
4.2.2 Login/Create Account Page



Portify – Automated Portfolio Generator



A screenshot of a web browser displaying the login page of 'Portify – Automated Portfolio Generator'. The browser's address bar shows 'localhost:44307/Home/Login'. The page features a dark background with a central modal form titled 'Welcome Back' and 'ENTER YOUR CREDENTIALS'. The form includes input fields for 'EMAIL' (containing 'admin@example.com') and 'PASSWORD' (masked with dots). A 'Forgot Password?' link is positioned to the right of the password field. A prominent blue 'Sign In' button is centered below the inputs. Below this, a section labeled 'OR CONTINUE WITH' contains a 'Sign in with Google' button. At the bottom of the modal, a link reads 'New here? Create Account'. The browser's top bar shows various icons and a bookmarks section with 'All Bookmarks'.



A screenshot of the same web browser displaying the registration page. The modal form is titled 'Welcome Back' and 'ENTER YOUR CREDENTIALS'. It features four input fields: 'FULL NAME' (containing 'John Doe'), 'EMAIL ADDRESS' (containing 'your@email.com'), 'PASSWORD' (masked with dots), and 'CONFIRM PASSWORD' (masked with dots). A blue 'Create Account' button is centered below these fields. Below the button, a section labeled 'OR SIGN UP WITH' contains a 'Sign up with Google' button. At the bottom of the modal, a link reads 'Already have an account? Sign In'. The browser's top bar and background are consistent with the previous screenshot.

4.2.3 Edit Profile Page

Portify

Home Explore Dashboard **My Profile**

Purvarajsinh Logout

SETTINGS

Edit Profile

EMAIL ADDRESS

purvarajsinh.test@gmail.com

IDENTITY LOCKED

FULL NAME

Purvarajsinh

☒ **Email Notifications**
Receive alerts when new templates are uploaded

Security

NEW PASSWORD (Optional)

Purvarajsinh

☒ **Email Notifications**
Receive alerts when new templates are uploaded

Security

NEW PASSWORD (Optional)

Leave blank to keep current

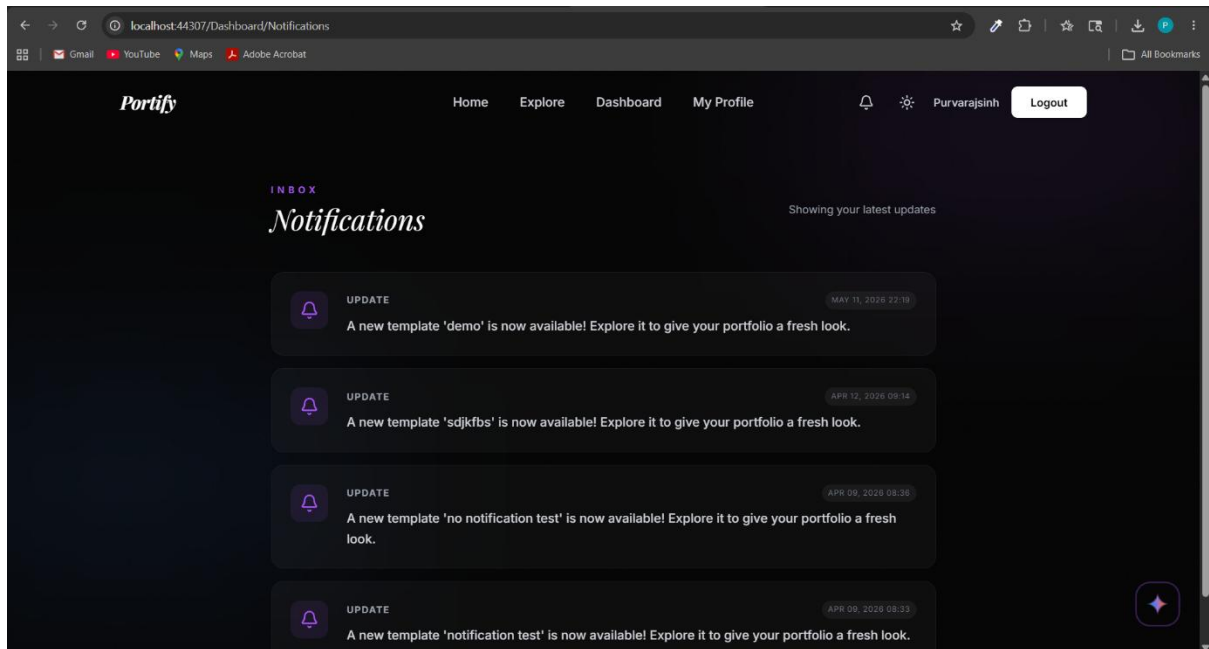
CURRENT PASSWORD *

Enter password to confirm changes

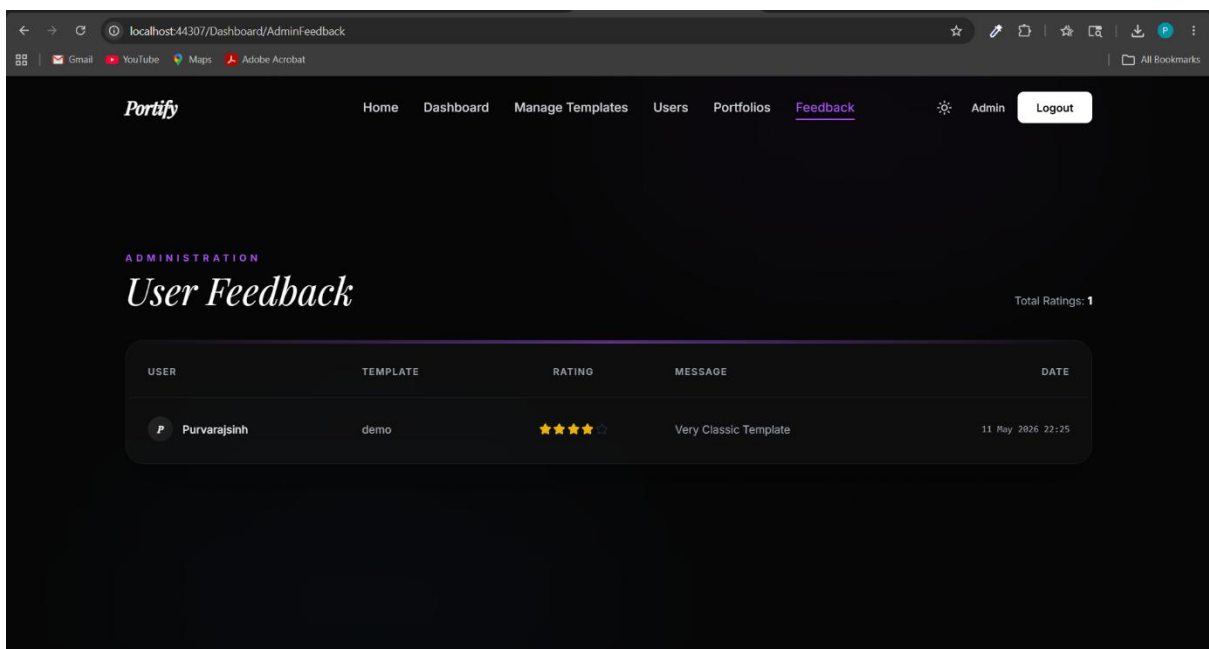
REQUIRED FOR VERIFICATION

CANCEL **SAVE CHANGES** →

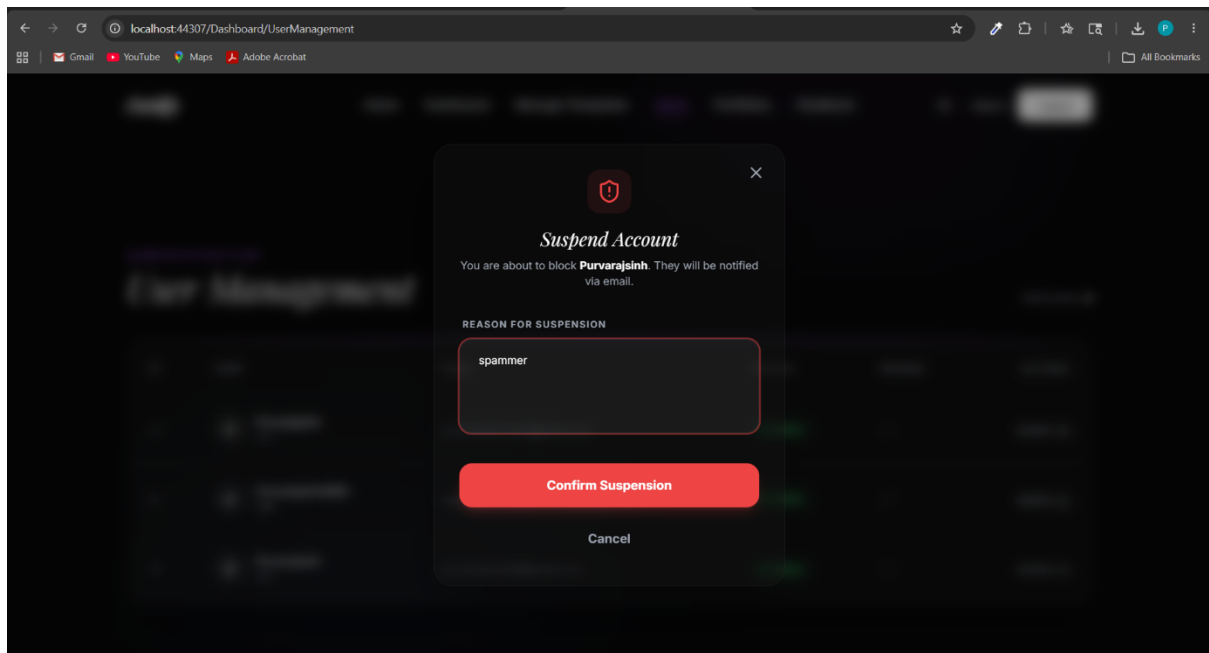
4.2.4 Notification Page



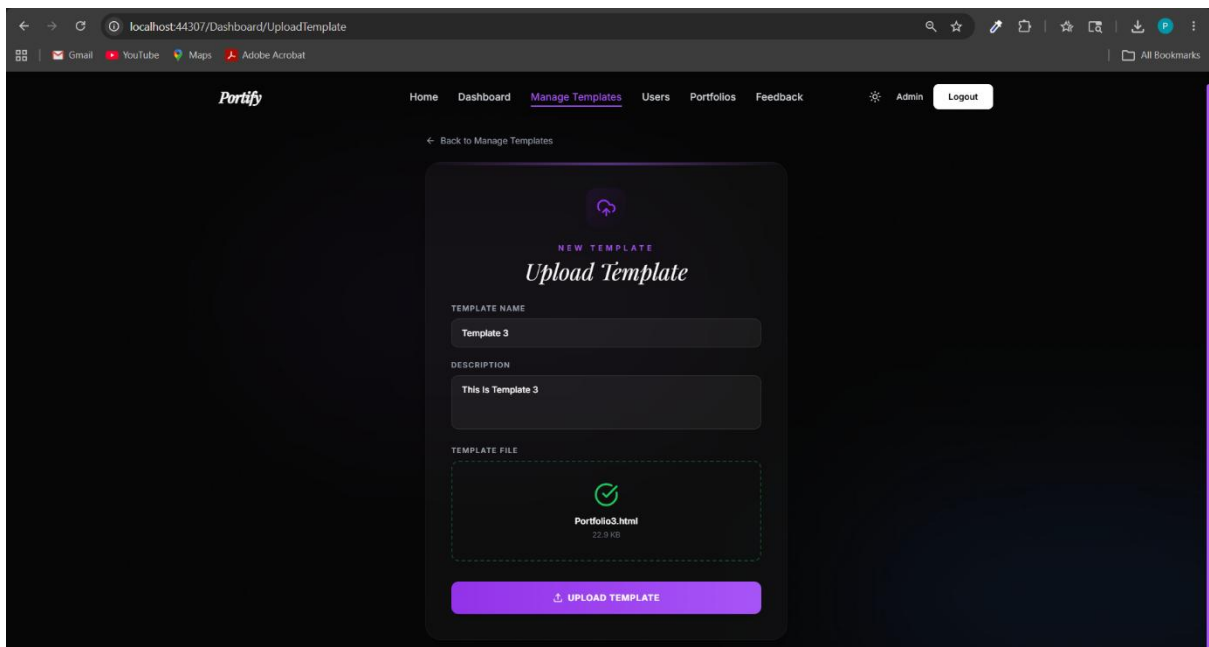
4.2.5 Feedback Page



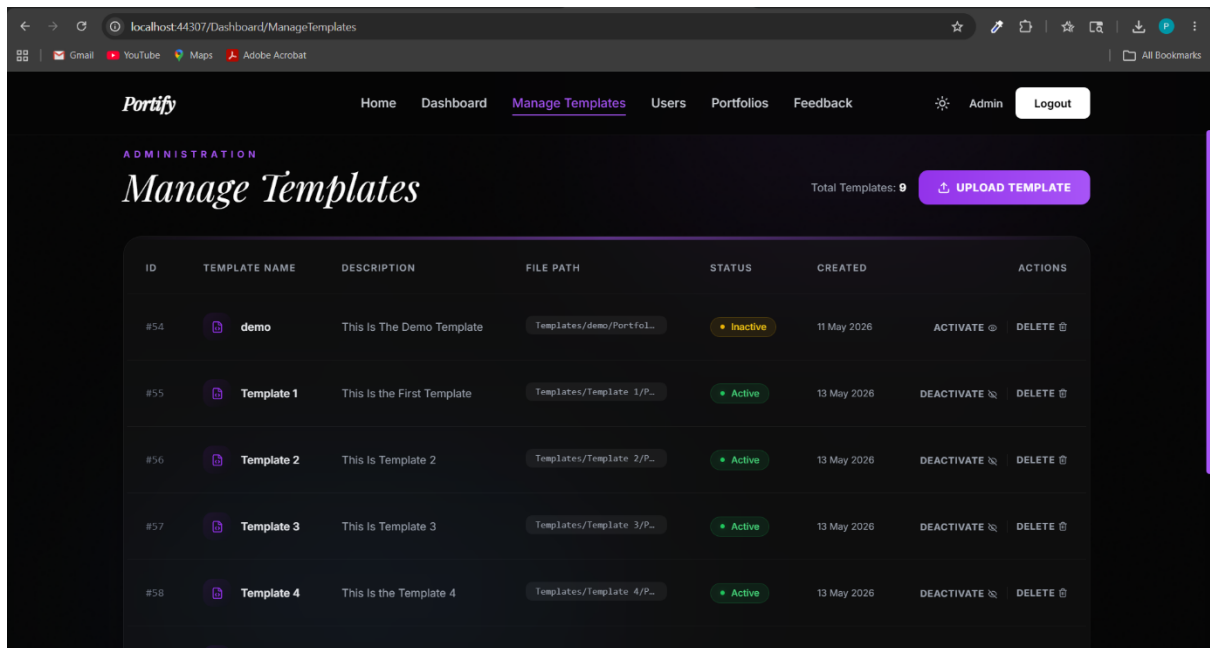
4.2.6 Suspend Account Page (Manage Account Page)



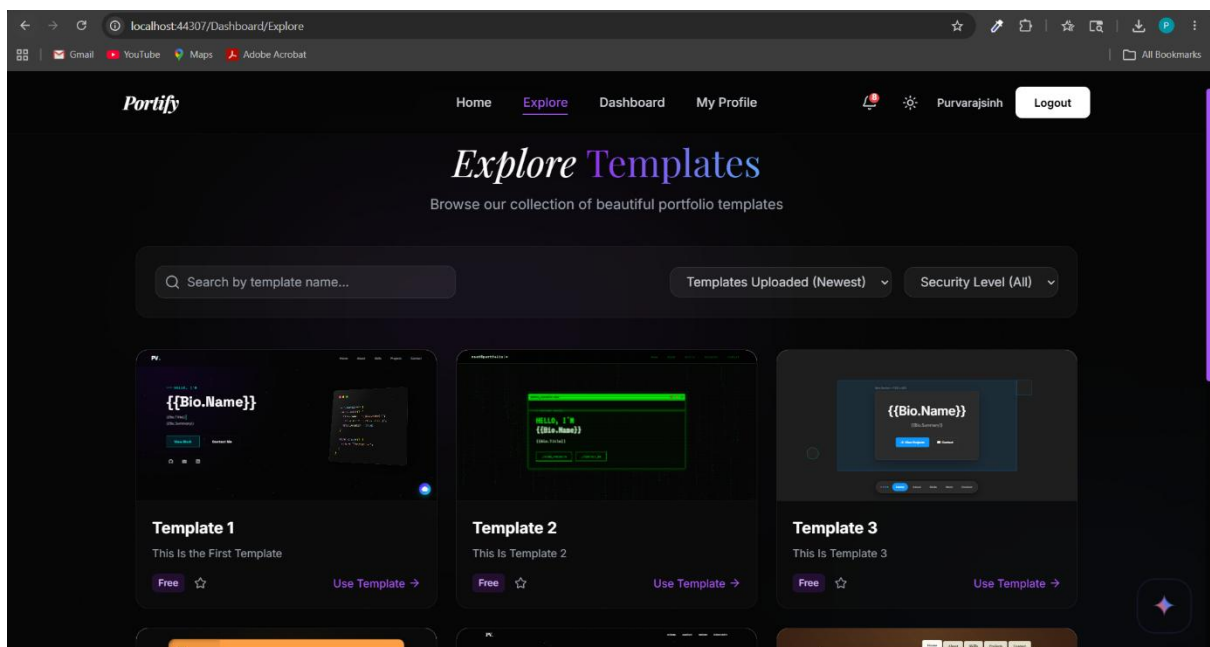
4.2.7 Upload Template Page



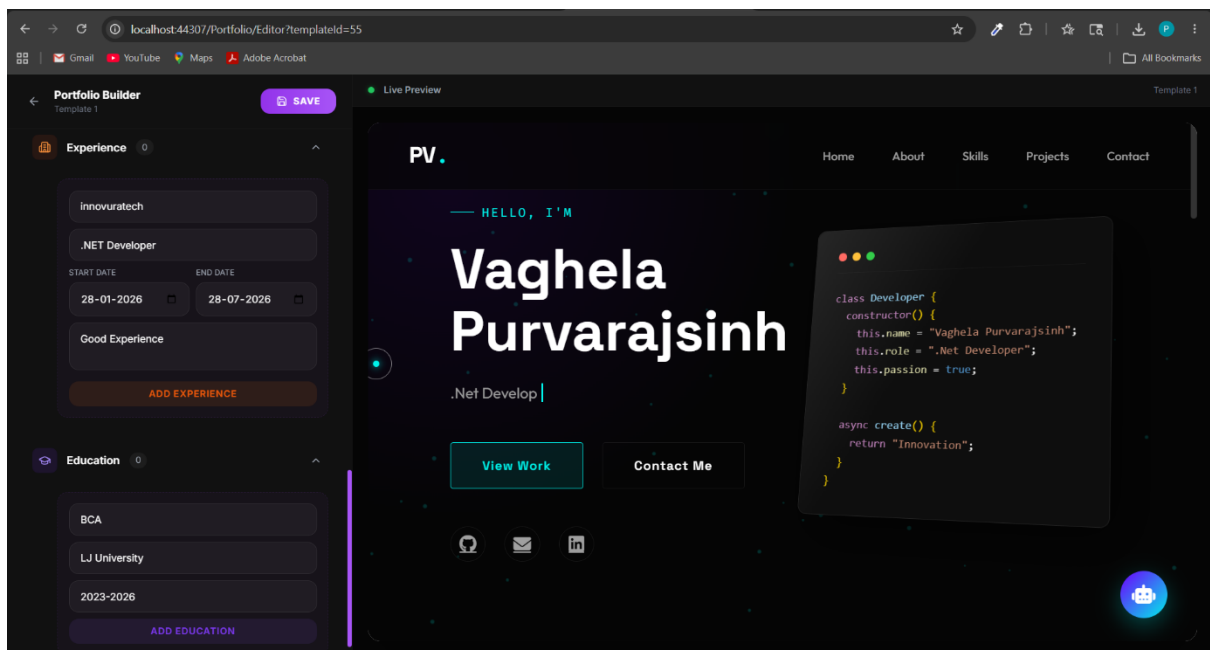
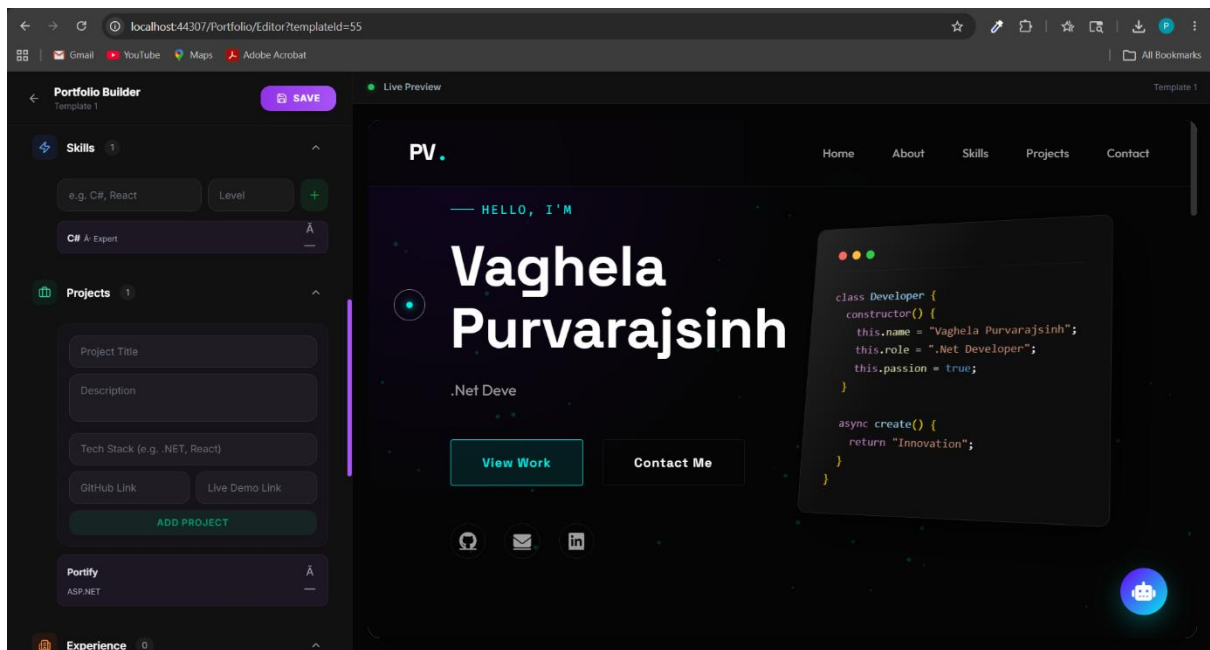
4.2.8 Manage Template Page



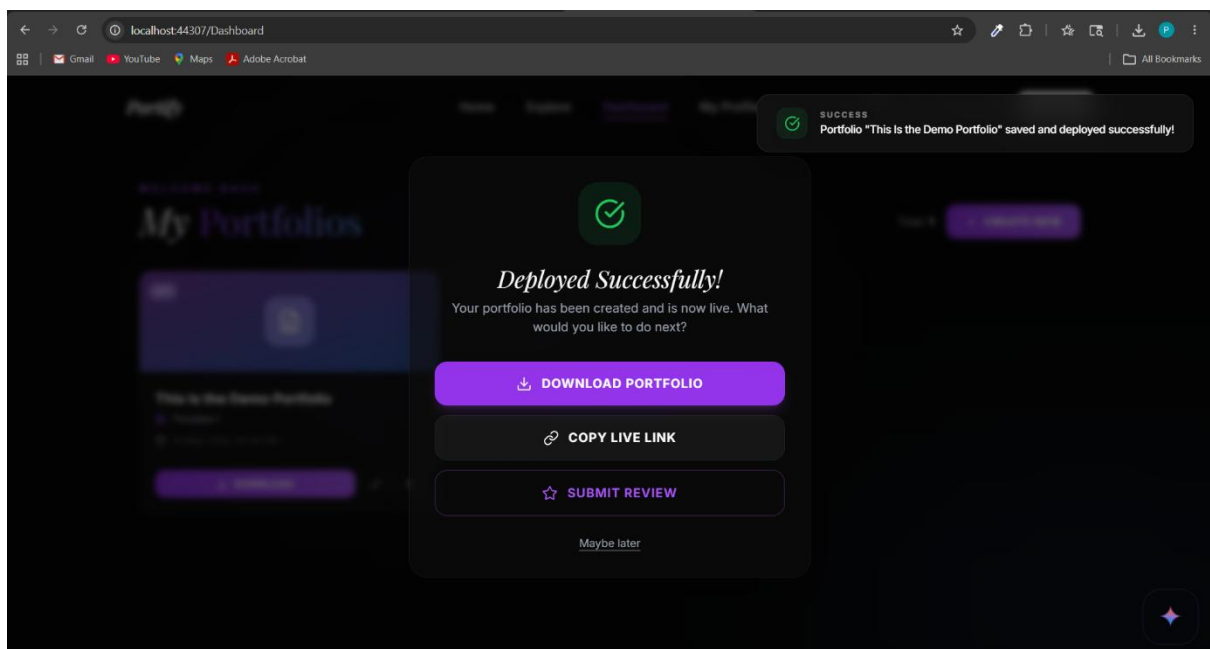
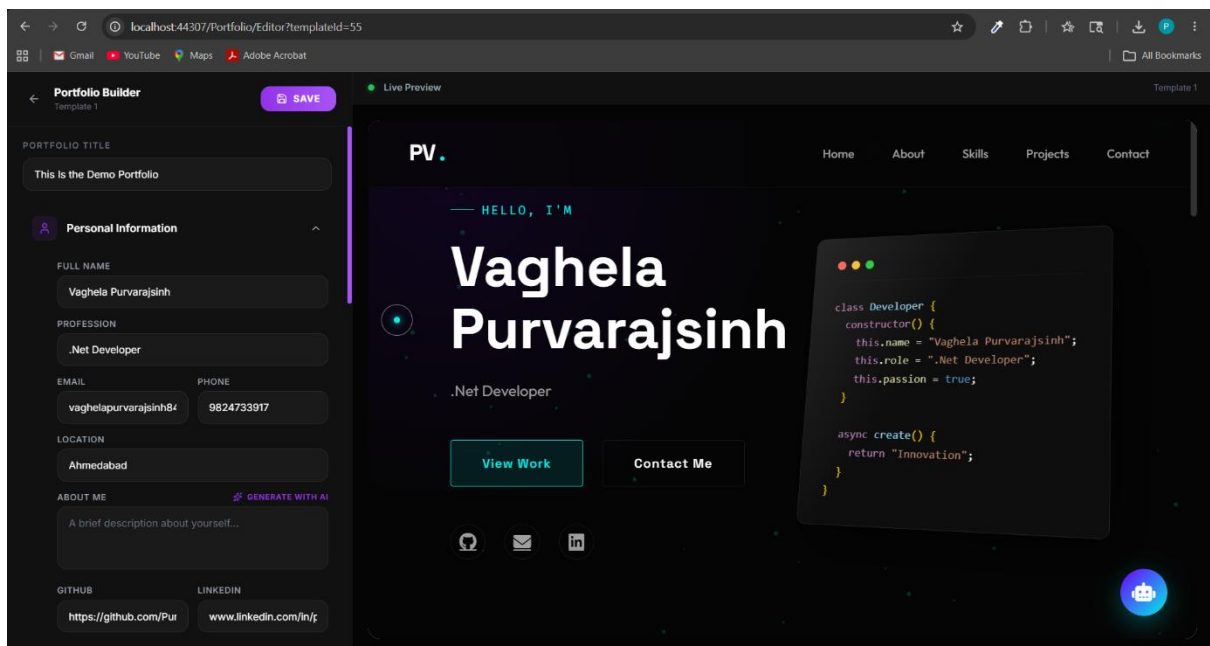
4.2.9 Explore Template Page



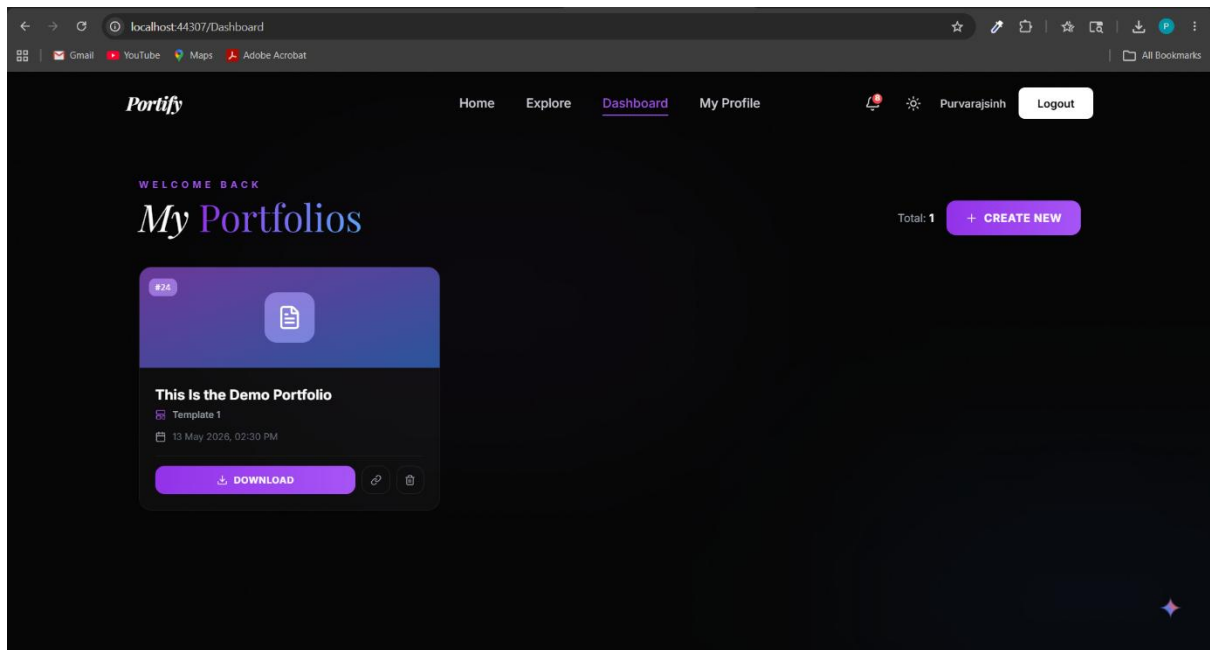
4.2.10 Create Portfolio Page



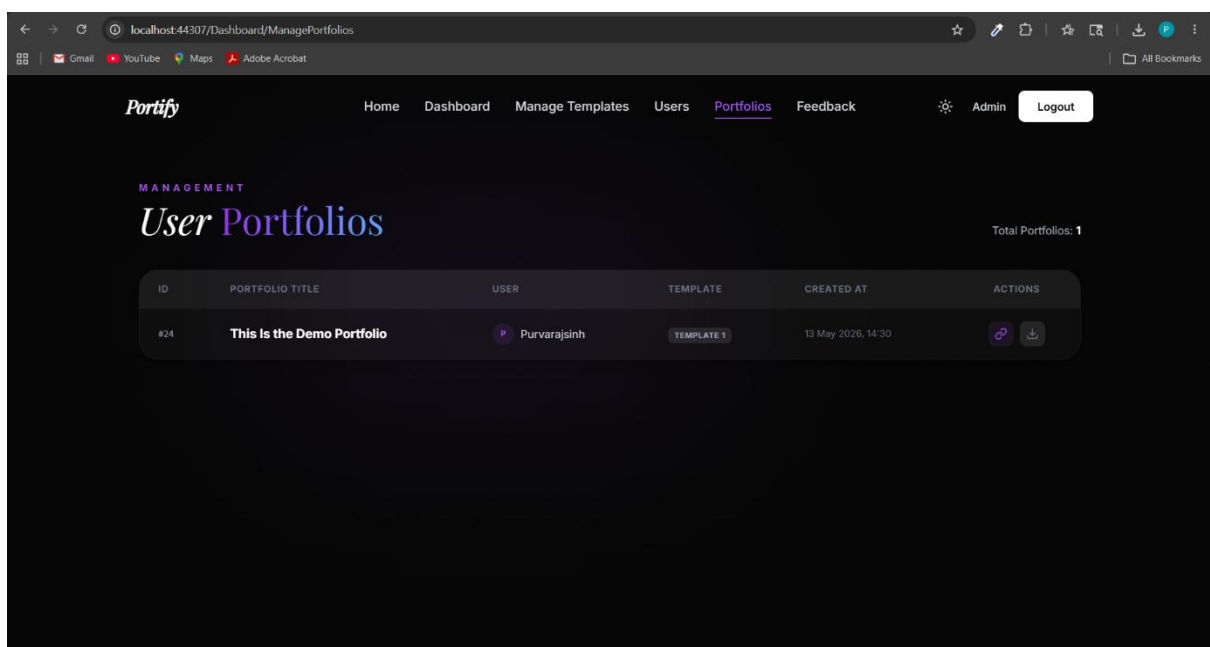
Portify – Automated Portfolio Generator



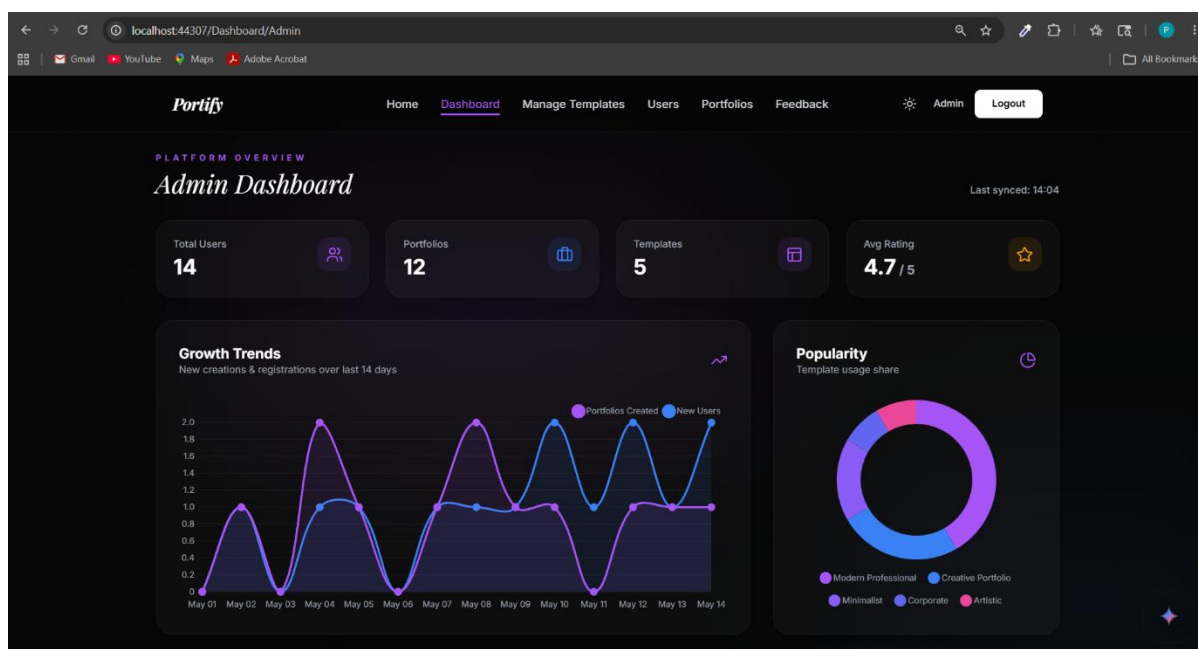
4.2.11 User Dashboard Page



4.2.12 Admin Portfolio Management Page



4.2.13 Admin Dashboard Page



5 Agile Documentation

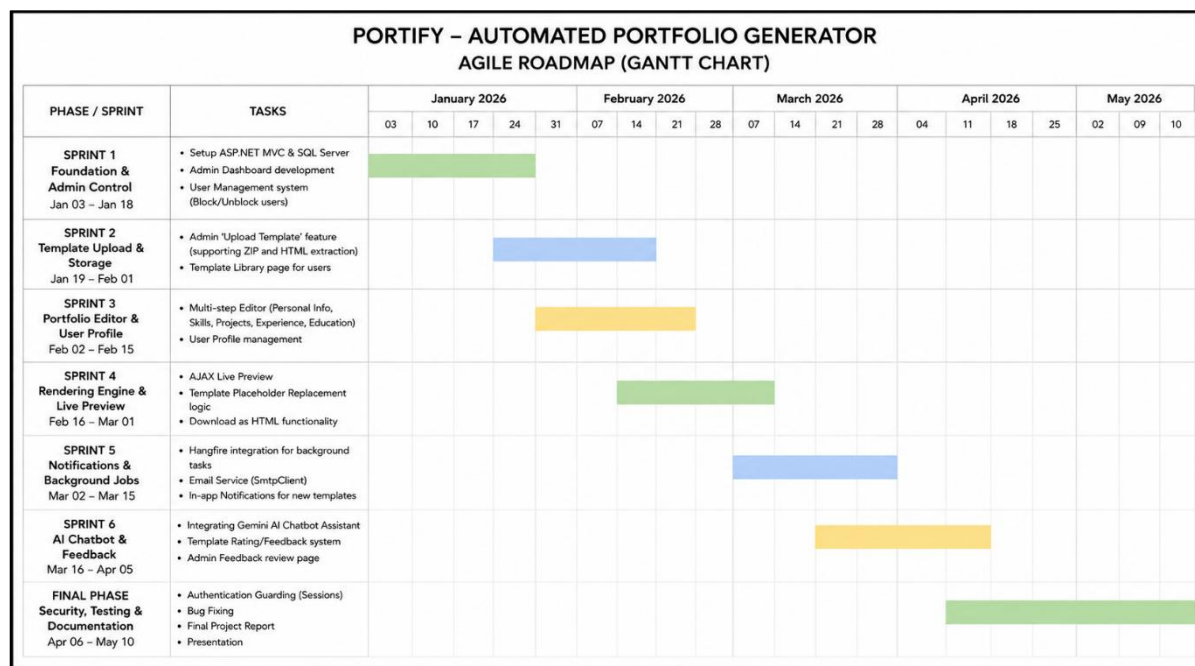
5.1 Agile Project Charter

Field Name	Details
Project Name	Portify - Automated Portfolio Generator
Project Champion	Vaghela Purvarajsinh
Project Sponsor	L.J Institute of Computer Application
Project Manager	Vaghela Purvarajsinh
Lead Developer	Vaghela Purvarajsinh
Key Stakeholders	L.J Institute of Computer Application, Students, and Young Professionals
Project Duration	03-01-2026 to 10-05-2026

Category	Category
Project Vision	To become the leading automated platform for generating high-quality, responsive portfolios, providing users with a seamless and efficient tool to build their professional digital presence.
Project Mission	To provide a seamless, automated platform for professionals and students to generate dynamic, customizable web portfolios by simply inputting their resume data and personal details.

Project Scope	The system includes user authentication, a dynamic portfolio editor (for personal, academic, and professional data), template management, a server-side rendering engine, and an export system to download standalone HTML portfolios. It also covers administrative control for managing users and templates.
----------------------	--

5.2 Agile Roadmap



5.3 Agile Plan

Task Name	Duration	Start	Finish	Status
Sprint #1: System Core & Admin Control	15d	03-01-2026	17-01-2026	Complete
Database Schema & Model Design	5d	03-01-2026	07-01-2026	Complete
Admin Dashboard Core UI	5d	08-01-2026	12-01-2026	Complete
User Authentication (Email/Google)	5d	13-01-2026	17-01-2026	Complete
Sprint #2: Template System & Uploads	14d	18-01-2026	31-01-2026	Complete
Designing HTML/CSS Templates	7d	18-01-2026	24-01-2026	Complete
Admin Template Upload Logic	7d	25-01-2026	31-01-2026	Complete
Sprint #3: Portfolio Editor & Profile	15d	01-02-2026	15-02-2026	Complete

User Profile Management	5d	01-02-2026	05-02-2026	Complete
Multi-step Portfolio Data Forms	10d	06-02-2026	15-02-2026	Complete
Sprint #4: Rendering & Preview Engine	14d	16-02-2026	01-03-2026	Complete
AJAX-based Live Preview System	7d	16-02-2026	22-02-2026	Complete
Template Placeholder Replacement Logic	7d	23-02-2026	01-03-2026	Complete
Sprint #5: Notifications & Background Jobs	14d	02-03-2026	15-03-2026	Complete
Hangfire Setup for Background Tasks	6d	02-03-2026	07-03-2026	Complete
Email Service & Notifications	8d	08-03-2026	15-03-2026	Complete
Sprint #6: AI Assistant & Feedback	14d	16-03-2026	29-03-2026	Complete
Gemini AI Chatbot Integration	9d	16-03-2026	24-03-2026	Complete
Template Feedback & Rating System	5d	25-03-2026	29-03-2026	Complete
Sprint #7: Final Testing & Deployment	41d	30-03-2026	10-05-2026	Complete
Security Audit & Session Guarding	10d	30-03-2026	08-04-2026	Complete
Final Bug Fixing & Performance Polish	15d	09-04-2026	23-04-2026	Complete
Project Documentation & Deployment	17d	24-04-2026	10-05-2026	Complete

5.4 Agile User Story (Minimum 3 Tasks)

User Story ID	As a (Type of User)	I Want to Perform	So That I Can
1	User	Select a professional template	Choose a design that fits my personal brand.
2	User	Enter my details (Skills, Projects, Experience)	Populate my portfolio without writing any HTML code.
3	User	See a Live Preview of my changes	Check how my website looks before saving or downloading it.

4	User	Download my portfolio as an HTML file	Host my professional website on my own domain or server.
5	User	Chat with the Portify AI Assistant	Get instant help and guidance on how to use the system.
6	Admin	Upload new templates in .ZIP format	Provide users with fresh and modern design options.
7	Admin	Manage and block users	Ensure the platform remains secure and free of misuse.
8	Admin	View user feedback and ratings	Understand which templates are popular and improve the system.

5.5 Agile Release Plan

Task Name	Duration	Start	Finish	Status	Release Date
Sprint #1: System Core & Admin Control	15d	03-01-2026	17-01-2026	Complete	17-01-2026
Database Schema & Setup	5d	03-01-2026	07-01-2026	Complete	07-01-2026
User Authentication Module	10d	08-01-2026	17-01-2026	Complete	17-01-2026
Sprint #2: Template System & Uploads	14d	18-01-2026	31-01-2026	Complete	31-01-2026
Admin Template Management UI	7d	18-01-2026	24-01-2026	Complete	24-01-2026
ZIP/HTML Upload & Extraction Logic	7d	25-01-2026	31-01-2026	Complete	31-01-2026
Sprint #3: Portfolio Editor & Profile	15d	01-02-2026	15-02-2026	Complete	15-02-2026
Portfolio Data Collection Forms	10d	01-02-2026	10-02-2026	Complete	10-02-2026
User Profile & Settings Management	5d	11-02-2026	15-02-2026	Complete	15-02-2026
Sprint #4: Rendering & Preview Engine	14d	16-02-2026	01-03-2026	Complete	01-03-2026
Live AJAX Preview System	7d	16-02-2026	22-02-2026	Complete	22-02-2026
HTML Rendering & Placeholder Engine	7d	23-02-2026	01-03-2026	Complete	01-03-2026
Sprint #5: Notifications & Jobs	14d	02-03-2026	15-03-2026	Complete	15-03-2026
Hangfire Background Email Service	8d	02-03-2026	09-03-2026	Complete	09-03-2026

In-App Notification System	6d	10-03-2026	15-03-2026	Complete	15-03-2026
Sprint #6: AI Assistant & Feedback	14d	16-03-2026	29-03-2026	Complete	29-03-2026
Gemini AI Chatbot Integration	9d	16-03-2026	24-03-2026	Complete	24-03-2026
Template Rating & Admin Feedback	5d	25-03-2026	29-03-2026	Complete	29-03-2026
Sprint #7: Final Launch & Testing	41d	30-03-2026	10-05-2026	Complete	10-05-2026
Bug Fixing & Security Guarding	20d	30-03-2026	18-04-2026	Complete	18-04-2026
Final Documentation & Project Release	22d	19-04-2026	10-05-2026	Complete	10-05-2026

5.6 Agile Spring Backlog

Task Name	Story	Sprint Ready	Priority	Status	Story Point
Sprint #1: System Core & Auth	Yes	Yes	High	Complete	10
Database Schema & Entity Framework	Yes	Yes	High	Complete	8
Secure Session Guarding & Auth	Yes	Yes	High	Complete	6
Sprint #2: Template Management	Yes	Yes	High	Complete	8
Admin Template ZIP Upload Logic	Yes	Yes	High	Complete	9
Template Explore Dashboard UI	Yes	Yes	Medium	Complete	4
Sprint #3: Portfolio Editor	Yes	Yes	High	Complete	10
Multi-tab Form (Skills/Projects)	Yes	Yes	High	Complete	8
JSON Data Serialization	Yes	Yes	High	Complete	5
Sprint #4: Rendering & Preview	Yes	Yes	High	Complete	10
AJAX-based Live Preview Engine	Yes	Yes	High	Complete	9
Placeholder Replacement Regex	Yes	Yes	High	Complete	7

Sprint #5: Notifications & Jobs	Yes	Yes	Medium	Complete	6
Hangfire Background Job Setup	Yes	Yes	Medium	Complete	7
Email Service Integration	Yes	Yes	Medium	Complete	5
Sprint #6: AI Assistant & Feedback	Yes	Yes	Medium	Complete	8
Gemini AI API Integration	Yes	Yes	Medium	Complete	9
User Feedback & Rating System	Yes	Yes	Low	Complete	3
Sprint #7: Launch & Documentation	Yes	Yes	High	Complete	5
Security Guarding & Error Handling	Yes	Yes	High	Complete	4
Final Documentation & Manual	Yes	Yes	High	Complete	5

5.7 Agile Test Plan

Project Name	Device
Portify	Windows 11 PC / Laptop

Field	Details
Test Case Id	01
Test Title	Create, Preview, and Download Portfolio
Module Name	Portfolio Management
Tested By	Vaghela Purvarajsinh
Priority	High
Execution Date	05-04-2026

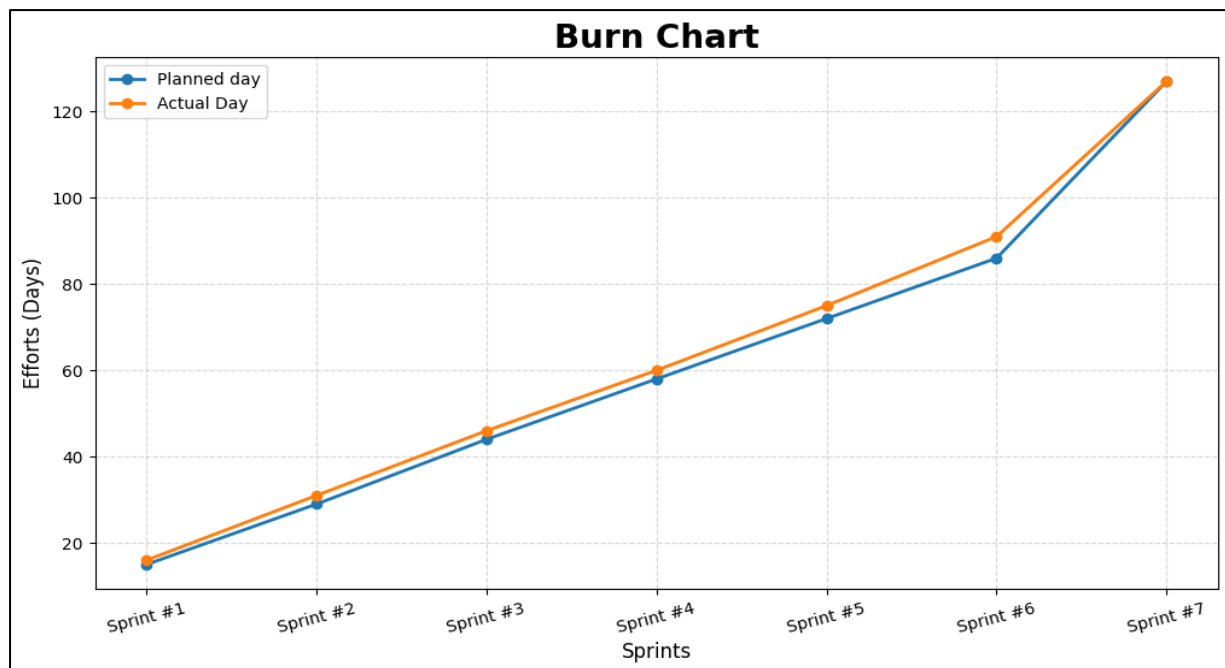
Test	Test Step	Action	Expected Result	Actual Result	Pass
1	Click on 'Explore Templates'	Open the template library	Template gallery should display all active designs	Gallery opened successfully with all templates	Yes

2	Select a Template and click 'Use'	Open the Portfolio Editor	Editor should open with forms for Skills, Projects, and Bio	Editor opened with correct form sections	Yes
3	Enter details and click 'Live Preview'	Trigger the AJAX Rendering Engine	The preview area should update instantly with user data	Preview updated in real-time with placeholders replaced	Yes
4	Click on the 'Save' button	Save portfolio data to database	Success message should appear and save to Dashboard	Portfolio saved successfully to User Dashboard	Yes
5	Click on the 'Download' button	Generate and download HTML file	A professional HTML/CSS file should download to the device	HTML file downloaded and opens perfectly in browser	Yes

Field	Details
Test Case Id	02
Test Title	AI Chatbot Interaction
Module Name	Chatbot Module
Tested By	Vaghela Purvarajsinh
Priority	Medium
Execution Date	06-04-2026

Test	Test Step	Action	Expected Result	Actual Result	Pass
1	Open Chatbot window	Click chatbot icon	Chat interface should pop up	Chat interface opened	Yes
2	Ask "How to download?"	Send message to Gemini AI	AI should provide simple, helpful instructions	AI replied with clear steps on downloading	Yes
3	Test without internet	Send any message	Error message should appear	Correct error shown: "Trouble connecting to brain"	Yes

5.8 Earned-value and burn charts



6 Proposed Enhancement

1. Export to PDF / Resume Format

- Currently, Portify generates a website (HTML). A major enhancement would be adding a "Download as PDF" feature that converts the web portfolio into a standard one-page resume format for physical job applications.

2. Portfolio Visitor Analytics Dashboard

- Add a tracking feature so users can see how many people have visited their portfolio link, which projects are clicked most, and the geographic location of the visitors (using Google Analytics or custom logic).

3. One-Click Cloud Deployment (Hosting)

- Instead of just downloading the files, add an integration with platforms like GitHub Pages, Netlify, or Vercel. This would allow users to "Publish" their portfolio to a live URL with a single click directly from the Portify Dashboard.

4. Custom Domain Mapping

- Allow professional users to map their own custom domains (e.g., 'www.yourname.com') to their Portify-hosted portfolio, making it a complete personal branding solution.

5. Collaborative Portfolio Review Mode

- Implement a "Review Link" feature where users can send a draft of their portfolio to mentors or friends. The reviewers could leave comments directly on the preview to suggest improvements before the final version is published.

6. Interactive 3D & WebGL Template

- Introduces highly interactive, next-generation 3D portfolio themes using WebGL and Three.js to help creative professionals stand out.

7 Conclusion

The development of **Portify** has been a significant journey in building a solution that bridges the gap between technical complexity and professional branding. In today's competitive job market, a digital presence is essential, yet many talented individuals lack the coding skills to build their own web portfolios. **Portify** successfully solves this problem by providing an automated, user-friendly platform where anyone can generate a high-quality, professional website in minutes.

Throughout this project, we have successfully implemented several advanced features, including:

- **Dynamic Rendering Engine:** Converting user data into fully functional HTML/CSS templates.
- **AI Integration:** Utilizing the **Gemini AI API** to provide an intelligent assistant for user support.
- **Secure Management:** A robust Admin panel for template and user oversight.
- **Agile Methodology:** Maintaining a disciplined development schedule from January to April 2026.

From a technical perspective, this project provided deep insights into the **ASP.NET MVC framework**, **SQL Server database management**, and **API integrations**. It also reinforced the importance of the **Agile Scrum** methodology in managing complex software lifecycles and delivering a polished final product.

In conclusion, **Portify** achieves its goal of being more than just a website builder; it is a career-enablement tool. The project stands as a complete, scalable, and efficient system that is ready for deployment and future enhancements, marking a successful milestone in my academic and professional development.

8 Bibliography

1. Books & Publications

- **Troelsen, A., & Japikse, P. (2021).** Pro C# 9 with .NET 5: Foundational Principles and Practices. Apress. (Used for C# logic and backend development).
- **Freeman, A. (2020).** Pro ASP.NET Core MVC 2. Apress. (Used for MVC architectural patterns and routing).
- **Lerman, J. (2016).** Programming Entity Framework. O'Reilly Media. (Used for Database-first approach and SQL Server integration).

2. Official Documentation

- **Microsoft Learn.** ASP.NET MVC Documentation. <https://learn.microsoft.com/en-us/aspnet/mvc/>
- **Google AI for Developers.** Gemini API Documentation. <https://ai.google.dev/docs> (Used for implementing the Chatbot Assistant).
- **Hangfire IO.** Background Processing for .NET. <https://www.hangfire.io/> (Used for notification and email services).
- **Mozilla Developer Network (MDN).** HTML, CSS, and JavaScript Web Docs. <https://developer.mozilla.org/>

3. Web Resources & Tools

- **W3Schools Online Web Tutorials.** HTML/CSS Templates and Responsive Design. <https://www.w3schools.com> (Used for portfolio template designing).
- **Stack Overflow.** Developer Community Q&A. <https://stackoverflow.com> (Used for troubleshooting MVC and AJAX errors).
- **Font Awesome.** Iconography and Styling Tools. <https://fontawesome.com> (Used for the user interface icons).
- **NuGet Package Manager.** Package Documentation for Newtonsoft.Json and Hangfire. <https://www.nuget.org>

4. Software & IDEs Used

- **Visual Studio 2022:** Integrated Development Environment for C# and ASP.NET.
- **Microsoft SQL Server Management Studio (SSMS):** Database management and query optimization.
- **Postman:** API testing for Gemini AI integration.